



Diplomarbeit

Bestellservice „HungerSatt“

Entwicklung und Design eines Online-Bestellservice für das Schul-
buffet

5AHINF-2025/26-DA05

Johannes Ellmer
Christos Magos

Betreuer

Mag. Robert Gröbl
Dipl. Ing. (FH) Dietmar Winkler

Ausgeführt im Schuljahr 2025/26

Kurzfassung

Abstract

Aufgabenstellung

Thema	Entwicklung und Design eines Online-Bestellservices für das Schulbuffet „HungerSatt“	
Aufgabenstellung (Kurzfassung)	Ziel ist die Planung, Entwicklung und das Design eines Online-Bestellservices für das Schulbuffet „HungerSatt“. Mithilfe der App können Schülerinnen und Schüler sowie Lehrkräfte Speisen und Getränke online vorbestellen, um Wartezeiten zu vermeiden. Zusätzlich ermöglicht die Anwendung dem Anbieter die Planung und Verwaltung von Menüs, die Einsicht in Bestellungen, die Verwaltung von Benutzerinnen und Benutzern sowie deren Guthaben.	
Kandidaten		Betreuer
Johannes Ellmer		Mag. Robert Gröbl
Christos Magos		Dipl. Ing (FH) Dietmar Winkler
Externer Kooperationspartner		
Firma: Schulbuffet „HungerSatt“ – Markus Gruber		
Auftraggeber: Markus Gruber		
Schriftliche Kooperationsvereinbarung liegt vor: Nein		
Budget: -		
Bedeckung durch: -		
Geplante Verwertung der Ergebnisse: Die Nutzung des Systems ermöglicht einen effizienteren Bestellvorgang. Der Buffetalltag wird viel übersichtlicher und alltägliche Prozesse werden optimiert. Außerdem können auf diese Weise lange Wartezeiten von Seiten der Kundinnen und Kunden vermieden werden. Dabei verfügt das System über hohe Sicherheitsstandards und ist kostengünstig zu betreiben.		

Eidesstattliche Erklärung

Die unterfertigten Kandidaten / Kandidatinnen haben gemäß § 34 (3) SchUG in Verbindung mit § 22 (1) Zi. 3 lit. b der Verordnung über die abschließenden Prüfungen in den berufsbildenden mittleren und höheren Schulen, BGBl. II Nr. 70 vom 24.02.2000 (Prüfungsordnung BMHS), die Ausarbeitung einer Diplomarbeit mit der umseitig angeführten Aufgabenstellung gewählt.

Die Kandidaten / Kandidatinnen nehmen zur Kenntnis, dass die Diplomarbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können.

Die Abgabe der vollständigen Diplomarbeit hat bis spätestens

07.04.2026

beim zuständigen Betreuer zu erfolgen.

Die Kandidaten / Kandidatinnen nehmen weiters zur Kenntnis, dass gemäß § 9 (6) der Prüfungsordnung BMHS nur der Schulleiter bis spätestens Ende des vorletzten Semesters den Abbruch einer Diplomarbeit anordnen kann, wenn diese aus nicht beim Prüfungskandidaten (bei den Prüfungskandidaten) gelegenen Gründen nicht fertiggestellt werden kann.

Kandidaten / Kandidatinnen	Unterschrift
Johannes Ellmer	
Christos Magos	

Gendererklärung

In dieser Diplomarbeit wird aus Gründen der besseren Lesbarkeit überwiegend die männliche Form verwendet. Dies geschieht ausdrücklich ohne die Absicht, andere Geschlechter oder Identitäten auszuschließen. Die gewählte Sprachform dient allein der Vereinfachung und folgt gängigen wissenschaftlichen Schreibkonventionen. Sämtliche Personenbezeichnungen sind daher geschlechtsneutral zu verstehen und beziehen alle Menschen gleichermaßen ein. Die Verwendung der männlichen Form impliziert keinerlei Wertung oder Diskriminierung.

Inhaltsverzeichnis

Kurzfassung	2
Abstract.....	2
Aufgabenstellung.....	3
Eidesstattliche Erklärung.....	4
Gendererklärung.....	5
Inhaltsverzeichnis	6
1 Planung.....	9
1.1 Aufgabenstellung	9
1.1.1 Projektfindung und Ausgangslage.....	9
1.1.2 Konzept und Rahmenbedingungen.....	10
1.1.3 Anforderungen und Ziele	11
1.2 Projektorganisation	15
1.2.1 Meilensteine und Zeitplan	15
1.2.2 Rollen- und Aufgabenverteilung	18
1.3 Stakeholder	21
1.3.1 Stakeholder-Analyse.....	21
1.3.2 Personas	23
1.4 Machbarkeitsstudien und Variantenbildung	25
1.4.1 Hosting mittels Proxmox-Server	25
1.4.2 Cloud-Dienste	25
1.4.3 Schlussfolgerung der Variantenbildung	27
2 Projektrecherche	28
2.1 Anwendungen	28
2.2 Technologien	29
2.2.1 Angular	29
2.2.2 NestJS	30
2.3 Programmiersprachen.....	32
2.3.1 Typescript	32
2.4 Datenschutz und Verifizierung.....	33
2.4.1 Datenschutz und DSGVO	33

2.4.2	Verifizierung und Authentifizierung.....	34
2.4.3	Zusammenfassung.....	36
2.5	PostgreSQL als relationale Datenbank.....	37
2.5.1	Relationale Datenbanken im Allgemeinen.....	37
2.5.2	PostgreSQL als DBMS	42
2.5.3	Zugriff auf PostgreSQL in containerisierten Umgebungen	43
2.5.4	Zusammenfassung.....	44
2.6	Grundlagen zur Microservice-Architektur	45
2.6.1	Grundlagen und wesentliche Merkmale.....	45
2.6.2	Kommunikation und Koordination von Microservices	46
2.6.3	Koordination in Microservice-Landschaften	46
2.6.4	Datenmanagement in Microservices	47
2.6.5	Kommunikation zwischen Frontend und Backend.....	48
2.6.6	Zusammenfassung.....	48
2.7	Containerisierung und Docker	49
2.7.1	Docker als Technologie der Containerisierung	49
2.7.2	Orchestrierung von containerisierten Microservices	50
2.7.3	Herausforderungen und Grenzen containerisierter Microservices.....	51
2.7.4	Zusammenfassung.....	51
2.8	Cloud Computing und Cloud-Plattformen	52
2.8.1	Grundlagen des Cloud Computing	52
2.8.2	Amazon Web Services vs. Microsoft Azure als Cloud-Anbieter.....	54
2.8.3	Cloud-Hosting-Alternativen.....	54
2.8.4	World4You als nationaler Hosting-Anbieter	55
2.8.5	Render als entwicklerfreundliche Cloud-Plattform	55
2.8.6	Gegenüberstellung mit etablierten Cloud Anbietern	56
2.8.7	Zusammenfassung.....	56
2.9	Einsatz sinnvoller Bibliotheken und Technologien	57
2.9.1	PDF-Generierung mit jsPDF.....	57
2.9.2	QR-Code-Erkennung mit html5-qrcode	58
2.9.3	Intuitive Bedienung mit Angular CDK Drag-and-Drop	58
2.9.4	Absicherung der Verifikation und Authentifizierung via E-Mail	59

2.9.5	Zuverlässiger E-Mail-Versand: SMTP und API-basierte Alternativen.....	60
2.9.6	Zusammenfassung.....	60
3	Umsetzung.....	61
3.1	Grundstruktur des Projekts	61
3.2	Frontend-Entwicklung	61
3.3	Backend und API-Schnittstellen	61
3.4	PostgreSQL DB und DB-Zugriffe	61
3.5	Deployment mit World4You	61
4	Sicherung der Softwarequalität (Testing).....	62
4.1	Frontend-Testing	62
4.2	Backend-Testing	62
5	Projektabschluss	63
5.1	Ergebnisse	63
5.2	Zukunftsausblick.....	63
5.3	Kurzanleitung zur Verwendung der Bestellapp.....	63
5.4	Danksagung	63
	Abbildungsverzeichnis	64
	Listingverzeichnis.....	64
	Abkürzungsverzeichnis.....	64
	Quellenverzeichnis.....	65

1 Planung

1.1 Aufgabenstellung

1.1.1 Projektfindung und Ausgangslage

Im schulischen Alltag stellt die Organisation von Bestellungen am Buffet eine immer wiederkehrende Herausforderung dar. Der bestehende Bestellprozess ist durch lange Wartezeiten, weite Wege zum Buffet sowie das Verwenden von handschriftlichen Zetteln geprägt. Diese Vorgänge sind nicht nur ineffizient, sondern erschweren auch eine strukturierte Erfassung und Auswertung der Bestellungen.

Aktuell werden Bestellungen ausschließlich persönlich und handschriftlich durchgeführt. Dies führt im Allgemeinen zu einem erhöhten Zeitaufwand, Unübersichtlichkeit und zusätzlichen Fehlerquellen. Darüber hinaus erschwert die fehlende digitale Unterstützung einen zeiteffizienten und transparenten Bestellprozess.

Genau hier kann eine digitale Lösung Abhilfe schaffen: Bestellungen werden einfach und schnell aufgenommen. Diese werden in Echtzeit erfasst und dem Buffetbetreiber unmittelbar bereitgestellt. Auf diese Weise werden Wartezeiten verkürzt, die Übersichtlichkeit des gesamten Bestellprozesses optimiert und der gesamte Ablauf wird deutlich vereinfacht und beschleunigt.

1.1.2 Konzept und Rahmenbedingungen

Benutzer / Zielgruppe

Die Bestell-App „HungerSatt“ richtet sich primär an die Kundinnen und Kunden des Schulbuffets „HungerSatt“. Unter die Zielgruppe fallen Schülerinnen und Schüler, Lehrerinnen und Lehrer sowie das Verwaltungspersonal. Auf der Bedienerseite führt die Anwendung beim Buffet-Inhaber selbst zur Effizienzsteigerung.

Allgemeine Zieldefinition

Ziel dieser Diplomarbeit ist die Entwicklung eines Online-Bestellsystems, das es den Kundinnen und Kunden des Schulbuffets ermöglicht, ihre Bestellungen bequem und schnell, beispielsweise direkt aus den Klassenzimmern, aufzugeben. Dabei sollten lange Warteschlangen, während der Pausen vermieden und gleichzeitig ein effizienterer, übersichtlicherer Bestellprozess sowie eine optimierte Buffetverwaltung gewährleistet werden.

Für das Schulbuffet bedeutet dies die Umsetzung und Implementierung eines Bestellsystems, mit dessen Hilfe Wartezeiten reduziert und der gesamte Buffetalltag effizienter gestaltet werden kann. Die Benutzer der App profitieren von einem schnellen und unkomplizierten Bestellvorgang, können Speisen komfortabel vorbestellen, und die Abwicklung sowie Speicherung ihrer Daten erfolgt unter höchsten Sicherheitsstandard und ist DSGVO-konform.

Systemvoraussetzungen

Aufgrund der relativ großen Zielgruppe wird die Bestell-App so entwickelt, dass sie von unterschiedlichen Geräten und Betriebssystemen bequem über den Browser gestartet werden kann. Wichtig ist auch der Backend-Aufbau, dieser soll ständig bereitstehen und dennoch kostengünstig sein. Das Frontend und Backend sollte also immer gleichermaßen erreichbar sein und wird daher über einen Proxmox-Server gehostet.

1.1.3 Anforderungen und Ziele

Die Bestell-App „HungerSatt“ ist eine innovative Anwendung, die einen effizienten und schnellen Bestellservice verspricht. Dabei profitieren Kunden von kürzeren Wartezeiten beim Bestellen und für das Buffetmanagement wird der gesamte Alltag optimiert.

Funktionelle Anforderungen

Muss-Ziele:

- **Benutzerfunktionen:**
 - **Sign-up, Sign-in:** Benutzer können sich mit E-Mail-Adresse und Passwort registrieren und anmelden. Dabei wird ihnen beim Registrieren automatisch eine Rolle zugewiesen. Die verwendete E-Mail wird automatisch über das dazugehörige Microsoft-Konto verifiziert.
 - **Menüübersicht:** User können Gerichte auswählen. Es gibt auch Filtermöglichkeiten (zB vegetarisch).
 - **Guthaben und Bestellsystem:** Kundinnen und Kunden verwenden ihr Guthaben, welches sie zuvor persönlich aufladen, um Bestellungen zu tätigen. Anschließend erhalten sie eine Bestellbestätigung. Das Guthaben können die Kundinnen und Kunden jederzeit online einsehen. Der gesamte Vorgang wird als Bestellung gespeichert und ist zusätzlich mit einem Loggingsystem ausgestattet.
 - **Warenkorb:** Die User können im Warenkorb noch einmal eine Zusammenfassung ihrer Bestellung ansehen - mit Mengenangaben, Zwischensummen und etwaigen Anmerkungen.
- **Inhaberfunktionen:**
 - **Dashboard:** Das Dashboard stellt dem Inhaber in Echtzeit sämtliche offenen Bestellungen übersichtlich dar. Dadurch ist jederzeit eine aktuelle Übersicht über den Bestellstatus gewährleistet.
 - **Menüplanung:** Über diese Funktion können Menüs erstellt, angepasst und verwaltet werden. Die erstellten Menüs sind anschließend für die Benutzer in der Menüübersicht einsehbar.

- **Gerichteverwaltung:** Einzelne Gerichte können detailliert definiert werden. Dabei werden zentrale Informationen wie Name, Preis, Allergene sowie die jeweilige Kategorie festgelegt.
- **Benutzerverwaltung:** Diese Funktion ermöglicht dem Inhaber einen umfassenden Überblick über alle aktiven Benutzerkonten. Zusätzlich besteht hier die Möglichkeit, das Guthaben der Kunden aufzuladen oder bei Bedarf Konten zu sperren.
- **Backend-Services & Datenbanken:**
 - Über eine REST-API werden Bestellungen, Benutzer, Gerichte und Zahlungen abgewickelt.
 - Die Daten werden in einer PostgreSQL-Datenbank persistent gespeichert.
 - Das gesamte System wird auf einem Proxmox-Server gehostet, daher muss dieser verwendet und implementiert werden.
- **Identity & Access Management:**
 - Das System ist rollenbasiert. Es besteht aus Inhaber, User und Admin Rollen. Das verwendete Konto wird überprüft und verifiziert.

Soll-Ziele

- **Benachrichtigungen** Kundinnen und Kunden erhalten eine Benachrichtigung, wenn eine Bestellung erfolgreich war oder neue Menüs ersichtlich sind. Der Inhaber bekommt ebenfalls eine Benachrichtigung, wenn eine Bestellung eingegangen ist.
- **Bestellbestätigung:** Der Inhaber soll eine Bestellung per QR-Code bestätigen und abschließen.
- **Finanzübersicht für den Anbieter:**
 - Umsatzstatistiken pro Tag/Woche/Monat: Der Inhaber selbst kann eine bestimmte Zeitspanne wählen, um den Gesamtumsatz einzusehen.
 - Die Finanzübersicht gibt einen Überblick über einzelne Bestellungen bzw. Bestellungskategorien.

Nicht-funktionelle Anforderungen

Soll-Ziele

- **Performance:** Das System soll auch bei hoher Nutzerauslastung noch gut ansprechbar sein.
- **Automatisierte Backups zur Datensicherheit**
 - Tägliche Backups der Datenbank
 - Notfall-Wiederherstellung < 2 Stunden
- **Deployment und Hosting:** Bereitstellung des Frontend und auch Verfügbarkeit des Backend über den Server
- **Aufbau des Backends entsprechend einer modernen Multi-Service Architektur**

Kann-Ziele

- **Zentrales Logging & Monitoring,** um etwaige Fehler überprüfen zu können
- **Progressive Web-App**

Risiken

Technische Risiken:

- Unerwartete Komplexität von Frontend und Backend
- Hosting über den Proxmox-Server kann kompliziert werden
- Probleme und Schwierigkeiten bei Datensicherung und Zugriffskontrolle (Keycloak)
- Abhängigkeit von Proxmox-Server
- Probleme mit Netzwerken und Firewall

Schätzung des Arbeitsaufwandes

Bereich	Aufgabe / Inhalt	Stunden
Frontend	Login / Registrierung	17
	Menüübersicht (Tages-/Wochenansicht, Filtermöglichkeiten)	20
	Gericht-Details & Warenkorb	23
	Benutzerverwaltung + Bestellbestätigung	25
	Dashboard	17
	Menüplaner CRUD & Gerichte verwalten (Name, Preis, Allergene, Kategorien)	36
Backend-Services	REST-API (Bestellungen, Benutzer, Gerichte, Zahlungen)	40
	PostgreSQL-Datenbankmodell, CRUD & Persistenz	30
	Business-Logik, Validierung, Services	20
Hosting	PostgreSQL-Datenbank und Container einrichten	35
Identity & Access Management	Keycloak Integration, Rollenmodell (Kunde, Inhaber, Admin), Token-Handling	25
Qualitätssicherung	Testphase, Unit-Tests, Integrationstests und Systemtests	30
Dokumentation	Recherche und vollständige Dokumentation des Projektes	60

1.2 Projektorganisation

1.2.1 Meilensteine und Zeitplan

Vorbereitungsphase	
Design und Entwurf	24.09.2025
Anfertigung eines Pflichtenheft	30.09.2025
Recherche	30.10.2025
Implementierung und Test	
Erstellung des Frontends mit Funktionen und Design	16.10.2025
API-Schnittstelle mit Backend und MultiService-Architektur	30.11.2025
Vorbereitungen für das Hosting	30.11.2025
Benutzer- und Rollenverwaltung	06.01.2026
Hosting der Microservicearchitektur	26.01.2026
Implementierung	
Frontend und Backend hosten und TestBuild	19.02.2026
Voraussichtliche Fertigstellung	04.03.2026

Gantt-Chart

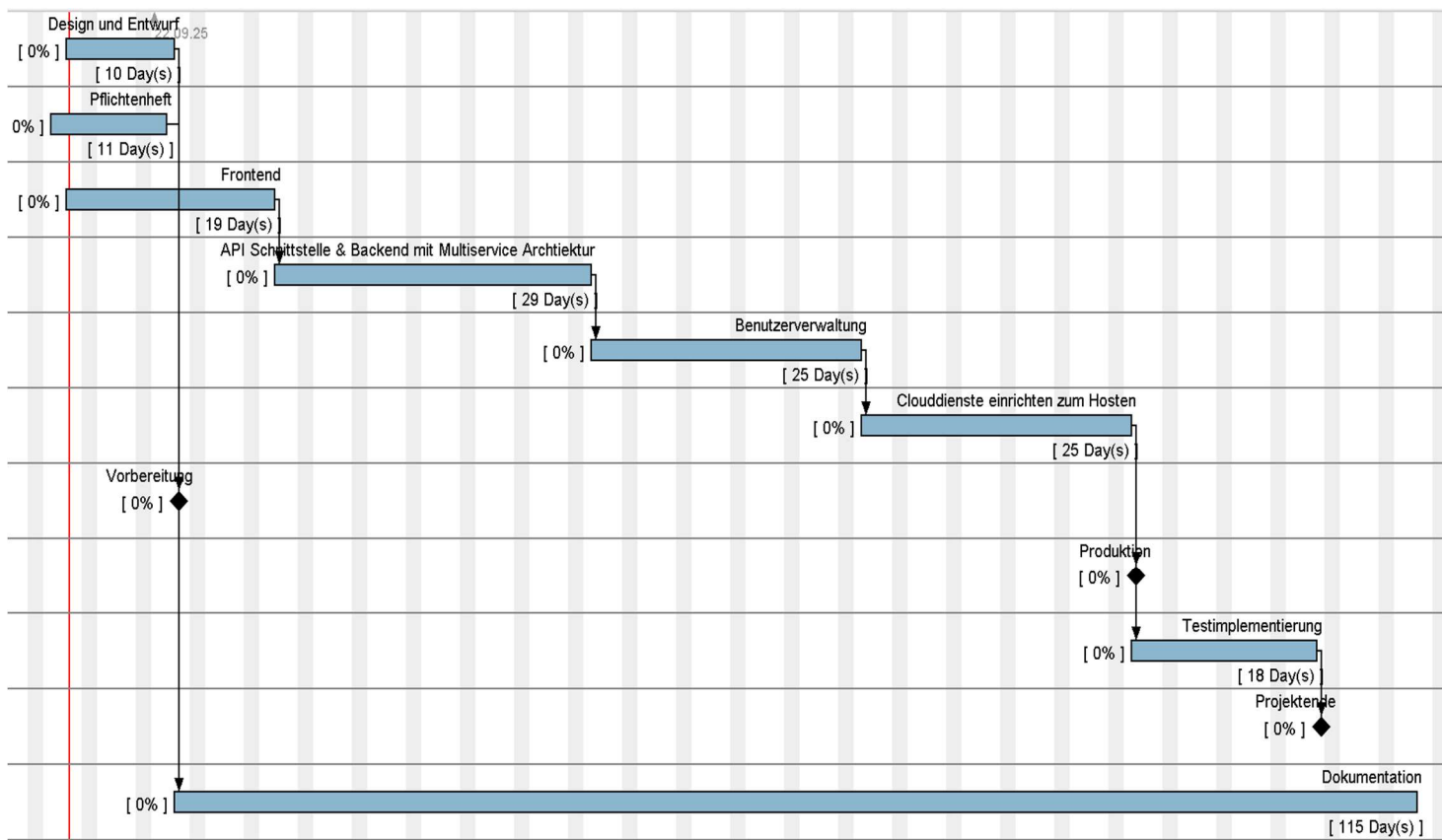


Abbildung 1: Zeitplan Gantt-Chart

Projektmeilensteine

Vorbereitungsphase

- **Design & Entwurf**
 - Erstellung von Mockups für alle Frontend-Seiten
 - Architektur und Design der Backend-Services (REST-API, Datenbankbindung)
- **Pflichtenheft (30.09.2025)**
 - Fertigstellung und Abgabe des Pflichtenhefts als verbindliche Grundlage
- **Recherche (30.10.2025):**
 - PostgreSQL, Datenschutz, Datensicherung und Rollenverwaltung (Keycloak), Proxmox Service
 - NestJS, API und Entwicklungsumgebung (IDE), Angular Framework, Proxmox und Deployment

Implementierung

- **Frontend (16.10.2025)**
 - Entwicklung aller Kernseiten mit Angular:
 - Login/Registrierung
 - Menüübersicht
 - Warenkorb mit Mengen, Zwischensumme, Checkout
 - Benutzerverwaltung für Admins (Rollenverwaltung)
 - Erste Tests auf Usability & Responsiveness
- **Backend-Anwendung (30.11.2025)**
 - API-Schnittstellen
 - Entwicklung der MultiService-Architektur mit NestJS
 - Anbindung an der PostgreSQL DB
 - Umsetzung von CRUD-Funktionen für Gerichte, Menüplaner und Bestellungen
- **Proxmox-Server aufsetzen und einrichten. (30.11.2025)**
- **Benutzer und Rollenverwaltung (06.01.2025)**
 - Einrichtung der Rollen Kunde, Inhaber, Admin

- **Vorbereitungen fürs Hosting (30.01.2026)**

Testphase und Pre-Production

- Testing
- Testlauf mit Fehlerbehebung

Projektabschluss

Projektablauf

HungerSatt Online-Bestellservice	Dauer in Stunden		Sep													Oct				Nov				Dec				Jän				Feb				Mar			
Aufgabe	Ellmer	Magos	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	1	2	3	4	5	6	7	8	9	10	11	12	13								
Anforderungen mit Markus besprechen	2	2																																					
Pflichtenheft erstellen und Präsentation	10	10																																					
Planung und Konzeption	5	5																																					
Planung Abschluss	17	17																																					
Recherche zu Technologien	10	10																																					
Entwicklung des Frontends	41	42																																					
Implementierung Menüpage	9	0																																					
Erstellung des Warenkorbs	8	0																																					
Dashboard	10	7																																					
Entwicklung des Guthabensystems	10	2																																					
Implementierung des Verwaltungssystems	0	25																																					
Aufbau der Frontendservices	4	8																																					
Entwicklung des Backends	30	32																																					
API Schnittstelle - Frontendbridge	0	10																																					
Multiservice-Architektur	0	15																																					
Proxmox Server aufsetzen und einrichten	15	5																																					
Datenbank aufsetzen	10	1																																					
Datenbank Bridge	5	1																																					
Rollen und Benutzerverwaltung	38	26																																					
Integration von Keycloak	8	6																																					
Datenschutz und Datensicherheit	15	10																																					
Implementation der Zugriffsverwaltung	15	10																																					
Hosting	15	11																																					
Hosting des Frontend	3	4																																					
Backend und DB hosten	12	7																																					
Abschluss	60	65																																					
Testing und Demolauf	18	23																																					
Dokumentation	40	40																																					
Summe	211	203																																					

Abbildung 2: Überblick Projektablauf

1.2.2 Rollen- und Aufgabenverteilung

Rollenverteilung

Auftraggeber	Schulbuffet „HungerSatt“– Markus Gruber
Hauptbetreuer	Mag. Robert Gröbl
Nebenbetreuer	Dipl. -Ing. (FH) Dietmar Winkler
Projektleiter	Johannes Ellmer
Projektteam	Christos Magos, Johannes Ellmer

Aufgabenverteilung

Johannes Ellmer

Dokumentation und Recherche zu: PostgreSQL-Datenbank, Datenschutz, Rollenverwaltung, Hosting

Frontend

- Entwicklung der Useransicht, die sich an den Endkunden richtet - dort können Nutzer nach Gerichten durchsuchen, filtern und Bestellungen tätigen.
- Implementierung eines Guthabensystems. Dies ermöglicht Kunden ihr Konto aufzuladen um anschließend Bestellungen abwickeln zu können.

Backend

- Architektur und Design der PostgreSQL-Datenbank und Entwicklung der Datenbank-Bridge, mit der die DB mit dem restlichen Backend kommuniziert. Außerdem erstellt das System regelmäßige Backups und Datensicherungen und verfügt über ein Loggingsystem.

Rollenverwaltung

- rollenbasierte Zugriffskontrollen mit Keycloak. Der Zugriff auf bestimmte Pages aber auch bestimmte Daten ist nur ausgewählten Rollen und Benutzern möglich.
- Datenschutz und Datensicherung: Daten werden verschlüsselt versendet und generell werden moderne Protokolle verwendet. Zusätzlich wird der Datenverkehr geloggt, um etwaige Fehler analysieren bzw. frühzeitig erkennen zu können. Datenbackups, zur Datensicherung.

Deployment

- Bereitstellung und Hosting des Backend

Christos Magos

Dokumentation und Recherche zu: NestJS und APIs, Angular Framework

Frontend

- Entwicklung der Ansicht für den Inhaber, in der alle grundlegenden Informationen wie offene Bestellungen, Menüplanung sowie Benutzer- und Guthabenverwaltung angezeigt werden. Außerdem ermöglicht das System einen übersichtlicheren und effizienteren Buffetalltag.

Backend

- Implementierung einer API-Schnittstelle zum Frontend sowie Ausarbeitung des Backends das aus einer Multiservice-Architektur besteht. Diese soll für eine bessere Skalierbarkeit sorgen.

Benutzerverwaltung

- Authentifizierung mit Microsoft-Konto und Sign-up / Sign-in

Deployment

- Das Frontend wird mit dem Server gehostet, um den Zugriff auf das Bestellservice aus verschiedenen Netzwerken, kostengünstig zu ermöglichen.

1.3 Stakeholder

Im Folgenden werden die wichtigsten Stakeholder des Projekts beschrieben, ihre Interessen und Ziele erläutert sowie ihr Einfluss auf den Projekterfolg analysiert.

1.3.1 Stakeholder-Analyse

- **Schüler:**

Die Schüler stellen die primäre Nutzergruppe dar. Sie wollen sich bequem im System von überall aus anmelden können. Dabei erfolgt die Anmeldung mithilfe des Microsoft-Kontos. Als Hauptinteresse gilt jedoch, dass die App bedienerfreundlich, intuitiv und zuverlässig ist. Der Einfluss auf das Projektziel ist in dieser Zielgruppe sehr hoch einzustufen, da die Nutzerzufriedenheit sich wesentlich auf den Erfolg der App auswirkt.

- **Lehrer:**

Lehrkräfte und anderes Schulpersonal stellen eine weitere Nutzergruppe da. Diese Gruppe hat ähnliche Ziele wie die Schüler. Trotz ihrer recht kleinen Mitgliederanzahl, ist die Auswirkung auf das Projektziel dennoch hoch einzuschätzen. Besonders ihre Vorbildrolle kann das allgemeine Feedback und das Nutzerverhalten beeinflussen.

- **Das Schulbuffet / Inhaber Markus Gruber:**

Das Buffet ist für die operative Umsetzung bekannt. Der Erfolg der App ist auch hier stark abhängig, wie strukturiert und effizient Abläufe erledigt werden können. Der Einfluss ist hier sehr hoch einzuschätzen, da das Schulbuffet der primäre Betreiber des Bestellsystems ist.

- **Entwickler:**

Das Entwicklerteam ist für die Konzeption und Implementierung der Anwendung verantwortlich. Die Qualität der Softwareentwicklung bestimmt im Wesentlichen den Projekterfolg.

- **Cloud Anbieter für das Hosting:**

Für das Hosten bzw. Bereitstellen der Software wird ein gängiger Cloudanbieter verwendet. Davon hängt zwar nicht der Projekterfolg selbst ab, jedoch ist die Zuverlässigkeit und uneingeschränkte Nutzbarkeit des Systems betroffen. Sollte es also

eine Störung beim Hosting geben, kann dies die Erreichbarkeit und damit die Funktionsfähigkeit der App erheblich beeinträchtigen.

Stakeholder-Matrix:

Als nächstes werden die identifizierten Stakeholder hinsichtlich ihres Einflusses auf das Projektziel sowie ihren Anteil am Projekterfolg eingeordnet. Die Kombination dieser beiden Dimensionen ermöglicht die Betrachtung und Einordnung der relevantesten Akteure.

Stakeholder	Interessen / Ziele	Einfluss	Interesse	Strategie / Umgang
Schüler:innen	Einfache Nutzung, schnelle Anmeldung, stabile App	Sehr hoch	Hoch	Feedback einholen, Usability-Tests, intuitive Oberfläche
Lehrkräfte & Schulpersonal	Reibungsloser Ablauf, einfache Integration	Hoch	Mittel	Einbinden, informieren, Feedbackrunden
Schulbuffet / Markus Gruber	Effiziente Abläufe, klare Bestellübersicht	Sehr hoch	Sehr hoch	Enge Zusammenarbeit, regelmäßiger Austausch
Microsoft Azure (Cloud)	Stabile Infrastruktur, Sicherheit	Mittel	Niedrig	Monitoring, SLA-Kontrolle

1.3.2 Personas

• Persona 1: Schüler – Lukas Steiner

Name: Lukas Steiner

Alter: 18 Jahre

Geschlecht: Männlich

Schulform: Höhere Technische Lehranstalt (HTL) für Informationstechnologie

Klassenstufe: 5. Jahrgang

Wohnort: St. Johann im Pongau

Technische Affinität: Sehr hoch

Lukas interessiert sich stark für Software, App-Design und digitale Innovationen. Er programmiert gelegentlich kleine Projekte und testet gerne neue Anwendungen. Er ist vertraut mit Cloud-Diensten wie Microsoft Azure, OneDrive und Teams.

Genutzte Geräte:

Smartphone, PC

Motivation / Ziele:

- Schneller, unkomplizierter Bestellvorgang über das Smartphone.
- Echtzeitübersicht über verfügbare Speisen und mögliche Abholzeiten.
- Modernes, responsives Design mit kurzen Ladezeiten.

Frustrationen:

- Lange Warteschlangen am Buffet.
- Langsame oder unübersichtliche Benutzeroberflächen.
- Fehlende mobile Optimierung bei Apps.

Persönliche Merkmale:

Lukas ist technikbegeistert, lösungsorientiert und erwartet von digitalen Anwendungen Professionalität und Komfort. Er ist offen für Neues, aber schnell kritisch, wenn die App nicht flüssig funktioniert.



Abbildung 3: Persona 1 (KI-generiert)

• Persona 2: Mag. Ingrid Bauer

Name: Mag. Ingrid Bauer

Alter: 62 Jahre

Geschlecht: Weiblich

Beruf: Lehrerin für Mathematik und Physik

Schule: Bundeshandelsakademie St. Johann (JOHAK)

Wohnort: St. Johann im Pongau

Technische Affinität: Gering bis mittel

Frau Bauer nutzt digitale Medien im Unterricht, ist jedoch keine begeisterte Technik-Anwenderin. Sie arbeitet mit



Abbildung 4: Persona 2 (KI-generiert)

Microsoft Teams und E-Mail, hat aber gelegentlich Schwierigkeiten mit neuen Apps und komplexen Benutzeroberflächen.

Genutzte Geräte: Laptop, Handy

Motivation / Ziele:

- Bequemer Zugriff auf das Buffetangebot während der Pausen.
- Zeitersparnis durch Vorbestellungen ohne Wartezeit.
- Einfache, gut lesbare Benutzeroberfläche ohne technische Hürden.

Frustrationen:

- Komplizierte oder unübersichtliche Apps.
- Fehlende technische Unterstützung.
- Wenn Funktionen nicht selbsterklärend sind.

Persönliche Merkmale:

Ingrid Bauer ist engagiert und strukturiert, legt Wert auf Zuverlässigkeit und Einfachheit. Sie steht digitalen Lösungen offen gegenüber, bevorzugt jedoch klare und stabile Systeme.

- **Persona 3: Buffetinhaber/Angestellte**

Alter: 45 Jahre

Geschlecht: Männlich

Beruf: Inhaber und Betreiber/Angestellter des Schulbuffets

Technische Affinität: Mittel

Herr Gruber nutzt digitale Kassensysteme und E-Mail regelmäßig, vermeidet aber unnötig komplexe Softwarelösungen. Er ist offen für Digitalisierung, sofern sie seinen Betrieb effizienter macht.

Genutzte Geräte:

Tablet (Android), Büro-PC mit Kassensystem, Smartphone

Motivation / Ziele:

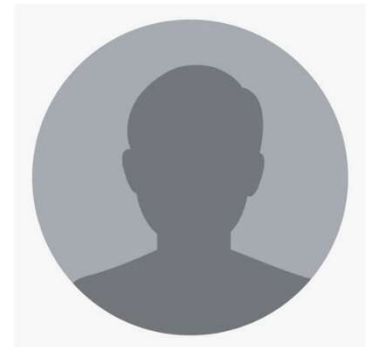
- Übersichtliche Darstellung der eingegangenen Bestellungen.
- Verlässliche Prognose für Einkaufs- und Vorbereitungsplanung.
- Minimierung von Wartezeiten und Fehlbestellungen.

Frustrationen:

- Technische Ausfälle während Stoßzeiten.
- Unübersichtliche oder fehlerhafte Bestelllisten.
- Fehlende Möglichkeit, Änderungen in Echtzeit einzusehen.

Persönliche Merkmale:

Markus Gruber ist praktisch orientiert, effizient und kundenbewusst. Er legt Wert auf funktionierende, verständliche Systeme, die Arbeitsprozesse vereinfachen und Zeit sparen.



1.4 Machbarkeitsstudien und Variantenbildung

1.4.1 Hosting mittels Proxmox-Server

Als erste Option wurde, geprüft, ob der Bestellservice (Frontend, Backend, PostgreSQL) auf dem schuleigenen Proxmox-Server gehostet werden kann. Technisch wäre der Betrieb über VMs bzw. Container möglich, die Anwendung könnte damit grundsätzlich in der geplanten Struktur bereitgestellt werden. [1]

Bewertung

In der Praxis scheitert der Ansatz an den Rahmenbedingungen des Schulnetzes: Firewall, Routing und Portfreigaben werden zentral verwaltet und können durch das Projekt nicht so angepasst werden, dass die Anwendung stabil und regulär von außen erreichbar ist. Damit wäre der Zugriff auf den Bestellservice im Regelfall nur innerhalb des Schulnetzwerkes möglich. Umgehungslösungen wie Tunnel sind zwar grundsätzlich denkbar, jedoch für eine größere Anzahl an Nutzern aus der HAK nicht geeignet. [1]

Ergebnis

Der Proxmox-Ansatz wird verworfen, da die notwendige externe Erreichbarkeit nicht zuverlässig bereitgestellt werden kann. [1]

1.4.2 Cloud-Dienste

Zielsetzung und Ausgangslage

Ziel dieser zweiten Machbarkeitsstudie ist die Bewertung einer Migration des Bestellservices in eine außerhalb des Schulnetzes betriebene Cloud-/Hosting-Umgebung. Untersucht werden Amazon Web Services (AWS), Microsoft Azure sowie als kostengünstigere Alternative World4You im Hinblick auf Kosten, Skalierbarkeit, Betriebseffizienz und Integrationsaufwand. Der finanzielle Rahmen ist mit maximal 10 € pro Monat vorgegeben.

Die aktuelle Architektur basiert auf einer Docker-Umgebung und umfasst:

- Frontend: Single-Page-Application (z. B. React/Vue)
- Backend-API: containerisierte Anwendung
- Datenbank: PostgreSQL in einem Docker-Container (Datenvolumen ≤ 1 GB)

Diese Struktur soll funktional beibehalten werden. Wo möglich, sollen Managed Services genutzt werden, um Betriebsaufgaben wie Patching, Monitoring und Skalierung zu reduzieren.

Anforderungen

Funktionale Anforderungen

- Bereitstellung der vier Kernkomponenten:
 - Frontend (SPA),
 - Backend-API,
 - PostgreSQL-Datenbank,
 - Keycloak-Instanz.
- Nahtlose Integration zwischen Frontend, Backend, Datenbank und Identity Management (z. B. über HTTP-APIs und standardisierte Authentifizierungsprotokolle wie OpenID Connect).
- Weiterverwendung der bestehenden Docker-Container (Backend, Keycloak), um den Migrationsaufwand zu reduzieren.

Nicht-funktionale Anforderungen

Kriterium	Vorgabe
Kosten	≤ 10 €/Monat
Skalierung	Auto-Skalierung bei Lastspitzen
Verfügbarkeit	Hoch während Geschäftszeiten
Wartung	Möglichst wenig Administration (Managed Services)

Variante 1: AWS

AWS stellt mit ECS und der serverlosen Containerlaufzeit Fargate eine verwaltete Ausführung für Container bereit, ohne dass eigene virtuelle Maschinen betrieben werden müssen. [2] Für PostgreSQL kann Amazon RDS als Managed-Datenbankdienst genutzt werden (Backups/Monitoring/Updates je nach Konfiguration und Serviceumfang). [2, 3]

Kostenbewertung

Für einen produktiven Betrieb entstehen typischerweise Fixkosten durch mindestens eine dauerhaft verfügbare Datenbankinstanz sowie Compute- und Netzwerkkomponenten. Eine Minimal-Konfiguration liegt grob ab ca. 30 € pro Monat und steigt je Laufzeit und Ressourcenbedarf häufig darüber. [2, 3]

Damit ist AWS im vorgegebenen Rahmen von max. 10 €/Monat in der Praxis nicht zuverlässig einzuhalten.

Variante 2: Azure Cloud

Azure stellt mit Azure App Service eine PaaS-Plattform bereit, auf der Webanwendungen und Container betrieben werden können. Welche Skalierungs- und Betriebsfunktionen verfügbar sind, hängt wesentlich vom gewählten App-Service-Plan (z. B. Basic/Standard/Premium) ab. [4–6] Für PostgreSQL steht mit Azure Database for PostgreSQL – Flexible Server ein verwalteter Datenbankdienst zur Verfügung, der u. a. Backups und Überwachung integriert. [4–6]

Kostenbewertung

In der betrachteten Konfiguration ergaben sich aufgrund der Plan-Mindestpreise sowie der Managed-PostgreSQL-Komponente Gesamtkosten in einer Größenordnung von rund 67 € pro Monat. [4–6]

Damit überschreitet Azure das vorgegebene Budget deutlich.

Variante 3: World4You

Als kostengünstigere Alternative bietet World4You vServer-Hosting, auf dem die bestehende Docker-Architektur weiterhin betrieben werden kann (Frontend, Backend, PostgreSQL, Keycloak als Container). Der Betrieb erfolgt dabei näher an klassischem IaaS: Die Anwendung bleibt containerbasiert, jedoch ohne die umfassenden Managed-Services-Funktionen großer Public-Cloud-Anbieter. [7, 8]

Kostenbewertung

Ein vServer kann im Rahmen des Budgets (≤ 10 €/Monat) realisiert werden. Im Gegensatz zu AWS/Azure sind jedoch mehr manuelle Betriebsaufgaben zu erwarten (z. B. Updates, Monitoring-Konzept, Backups), da keine vollumfänglichen Managed-Database-/PaaS-Komponenten genutzt werden. [7, 8]

1.4.3 Schlussfolgerung der Variantenbildung

Die Variantenbildung zeigt, dass ein Hosting auf dem schuleigenen Proxmox-Server zwar technisch umsetzbar wäre, jedoch an der fehlenden externen Erreichbarkeit aufgrund der Rahmenbedingungen des Bildungsnetzes scheitert und daher verworfen wird.

Bei den Cloud-Diensten erfüllen AWS und Azure die technischen Anforderungen (z. B. Managed Services, Skalierung), überschreiten jedoch die Kostengrenze deutlich und kommen damit wirtschaftlich nicht in Frage.

Als Lösung bleibt World4You: Der vServer ermöglicht den Betrieb außerhalb des Schulnetzes innerhalb des Kostenrahmens und erlaubt die Weiterverwendung der bestehenden Docker-Container. Im Vergleich zu den großen Cloud-Anbietern ist dafür jedoch ein höherer manueller Betriebsaufwand (z. B. Updates/Backups/Monitoring) einzuplanen.

2 Projektrecherche

Im folgenden Abschnitt werden die verwendeten Technologien genauer beschrieben und elementare Themen erläutert. Ziel ist es, dem Leser die verwendeten Technologien und Komponenten des Projektes zu vermitteln, darunter auch jene Bestandteile, die für den Anwender später möglicherweise nicht mehr sichtbar sind. Darüber hinaus fließen in diesem Kapitel alle im Rahmen der Recherche für diese Diplomarbeit gewonnenen Erkenntnisse ein, um ein allgemeines, besseres Verständnis für das gesamte Thema zu schaffen.

2.1 Anwendungen

Im Rahmen dieser Diplomarbeit kamen verschiedene Anwendungen und Systeme zum Einsatz. Sowohl im Bereich der Softwareentwicklung als auch im Projektmanagement leisteten alle verwendeten Programme einen wesentlichen Beitrag zum Erfolg des Projekts. Im Folgenden werden diese Anwendungen und Systeme näher vorgestellt.

2.2 Technologien

2.2.1 Angular

Im Rahmen dieser Diplomarbeit wird eine webbasierte Applikation umgesetzt, die von Endbenutzern sowie dem Buffetinhaber selbst verwendet werden soll. Es folgen daher besondere Anforderungen an die Benutzeroberfläche, wie eine übersichtliche Darstellung der Gerichte, eine einfache Verwendbarkeit auf verschiedenen Endgeräten, sowie die Trennung der Funktionen mithilfe der Verwendung von Rollen. Zusätzlich soll die Anwendung stabil und wartbar sein, sowie für zukünftige Erweiterungen entworfen werden.

Für die Erfüllung dieser Anforderungen ist ein geeignetes Webframework notwendig, das die strukturierte Entwicklung einer komplexen Benutzeroberfläche, sowie eine zuverlässige Anbindung an das Backend ermöglicht. Die Entscheidung für diese Technologie fiel dabei auf Angular. Im folgenden Kapitel werden die relevanten Grundlagen, Konzepte und Eigenschaften des Frameworks erläutert, um diese Entscheidung nachvollziehbar zu begründen.

Angular ist ein Webframework zur Entwicklung sogenannter Single Page Applications (SPA). Es ist von Google entwickelt und basiert auf der Programmiersprache Typescript. Das Ziel und der hauptsächliche Vorteil von Angular ist die erhöhte Übersichtlichkeit von großen und komplizierten Webanwendungen, wodurch eine bessere Wartbarkeit und langfristige Erweiterbarkeit ermöglicht wird. Aufgrund dieser Eigenschaften des Frameworks wurde Angular als Framework für die Frontend Entwicklung des Projekts ausgewählt. [9, 10]

Angular folgt einem komponentenbasierten Architekturansatz. Jede Komponente beinhaltet jeweils eine Typescript Datei. Diese Datei ist für die Logik zuständig. Eine HTML-Datei kontrolliert die Ansicht der Datei und mit einer CSS-Datei kann das Design zusätzlich bearbeitet werden. Diese Trennung von Logik und Design macht die Funktionsweise der Website übersichtlicher und vereinfacht den Entwicklungsprozess. Im Kontext des Bestellsystems ist der komponentenbasierte Ansatz von großem Vorteil, da die Entwicklung von Seiten mit einem komplizierten Aufbau, wie zum Beispiel der Menüplan, ohne einen solchen Ansatz, sehr zeitaufwändig wäre. Auch für die Übersicht der zahlreichen Seiten des Systems ist dieser Architekturansatz von Vorteil. Dies ermöglicht das Wechseln von Ansichten, ohne das vollständige neu Laden der Website. [9, 10]

Ein zugehöriger URL-Pfad kann für jede Komponente definiert werden. Diese Pfade werden dann verwendet, um zwischen den Komponenten der Applikation zu wechseln. Die Zugriffskontrolle für die jeweiligen Komponenten lässt sich mit sogenannten Route Guards implementieren. Damit können Rollen definiert werden, die Zugang zu einer bestimmten Komponente haben soll. Diese Funktionalität ist für die Umsetzung der Rollenbasierten Zugriffskontrolle des Bestellsystems essenziell. Mithilfe der Route Guards können die Rollen

Kunde und Inhaber definiert werden damit sichergestellt wird, dass jeweils die richtigen Komponenten zur Verfügung gestellt werden.[10]

Die Kommunikation mit dem Backend erfolgt mittels dem HttpClient von Angular. Mit diesen können Daten im JSON-Format zwischen Frontend und Backend ausgetauscht werden. Damit können beispielsweise Authentifizierungsinformationen an das Backend übertragen werden. Im Rahmen dieser Diplomarbeit wird HttpClient für die Verbindung mit den, mit NestJS implementierten, API-Gateway genutzt. [11]

Insgesamt stellt Angular für die Entwicklung des Frontends der Bestellapplikation einen geeigneten technologischen Grundsatz. Es lässt die klare Trennung von Logik und Ansicht zu, bietet ein integriertes Navigationssystem und ermöglicht die Kommunikation mit dem Backend. Diese Eigenschaften sind für größere Systeme mit mehreren Rollen von großer Bedeutung.

2.2.2 NestJS

Neben dem Frontend wird für die Entwicklung des Bestellsystems auch ein geeignetes Backend benötigt. Dieses Backend soll eine zuverlässige Schnittstelle zum Frontend bereitstellen. Die hauptsächliche Arbeit davon ist die Verarbeitung von Bestellungen, Speisekarten und Benutzerdaten. Aspekte wie Erweiterbarkeit und Wartbarkeit müssen auch berücksichtigt werden.

Für die Erfüllung dieser Anforderungen wird ein Framework benötigt, welches die Entwicklung einer Web-API ermöglicht und für die Implementation einer Multiservicearchitektur geeignet ist. Für die Umsetzung des Backends wurde daher NestJS gewählt. Im folgenden Kapitel werden die grundlegenden Konzepte von NestJS sowie dessen Bedeutung im Kontext des Backends des Bestellsystems erläutert. [12]

Grundlagen und Architektur

NestJS ist ein serverseitiges Webframework für die Entwicklung von Webanwendungen und APIs. Es basiert auf NodeJS und verwendet intern Technologien wie Express und Fastify. Ein zentrales Merkmal von NodeJS ist die Unterstützung von Typescript, wodurch eine gut strukturierte Codebasis entsteht. [12]

Bei der Architektur von NestJS liegt der Fokus auf Modulen. Eine Anwendung besteht aus verschiedenen Modulen. Jedes dieser Module ist für einen bestimmten Funktionsbereich zuständig. Jedes Modul beinhaltet Controller und Services. Controller verarbeiten eingehende HTTP-Anfragen und Services sind für die eigentliche Geschäftslogik zuständig. Diese Trennung der Aufgaben macht die Anwendung wartbar und unterstützt eine klare Verantwortungsverteilung. [13]

Das Dependency-Injection-System von NestJS bildet auch einen zentralen Bestandteil seiner Architektur. Damit werden Abhängigkeiten wie Services und Datenzugriffe automatisch

verwaltet und zur Verfügung gestellt. Dank dieses Systems können Komponenten unabhängig voneinander entwickelt und erweitert werden. [13]

Ein zentrales Ziel bei dem Aufbau des Backends ist ein Multiservice-orientierter Architekturan-satz, also die Trennung der einzelnen Funktionsbereiche. Die modulare Architektur von NestJS eignet sich für die Erfüllung dieses Ziels sehr gut. Die verschiedenen funktionale Bereiche des Projekts, wie Benutzerverwaltung, Bestellungen oder Menüverwaltung können in Modulen oder Services umgesetzt werden. Damit erfolgt eine klare Trennung dieser Funktionen, was die Grundlage einer Multiservice Architektur bildet. Diese Struktur verbessert die Übersichtlichkeit des Systems und erleichtert somit auch zukünftige Erweiterungen.

Kommunikation mit dem Frontend

NestJS eignet sich besonders für die Entwicklung von REST-basierten APIs. HTTP-Endpunkte können mit der Verwendung von sogenannten Decorators klar definiert werden. Damit können GET-, POST-, PUT- und DELETE- Anfragen implementiert werden. Diese Endpoints ermöglichen die Kommunikation zwischen Frontend und Backend bzw. den Austausch von Daten im JSON-Format. [12]

Für die Funktionalität der Bestellapplikation stellt NestJS die API bereit, die dem Frontend ermöglicht, Informationen wie Bestellungen, Menüs und Benutzerdaten abzurufen. Eine strukturierte Definition der Endpunkte ist dabei wichtig, um den Aufbau der Kommunikation zwischen den beiden Systemen so klar nachvollziehbar wie möglich zu machen. [12]

Zusammenfassung

NestJS ist ein leistungsfähiges und strukturiertes Framework für die Entwicklung von modernen backend Anwendungen. Der Modulare Aufbau und die klare Trennung von Controller und Services machen das Framework zu einem geeigneten System für die Entwicklung größerer Webanwendungen mit komplizierter Logik.

Aufgrund dieser Eigenschaften bildet NestJS im Rahmen dieses Projekts die Grundlage für die serverseitige Verarbeitung aller zentralen Funktionen und ermöglicht eine saubere, skalierbare und wartbare Umsetzung der REST-API.

2.3 Programmiersprachen

2.3.1 Typescript

Es werden im Frontend als auch im Backend Webframeworks verwendet, die auf die Programmiersprache Typescript basieren. Um die verwendeten Technologien besser einordnen zu können ist es wichtig die Grundlagen von Typescript sowie dessen Bedeutung im Projektkontext zu betrachten.

Typescript ist eine von Microsoft entwickelte Programmiersprache, die auf Javascript aufbaut und dieses erweitert. Der Typescript-Code wird nicht direkt im Browser ausgeführt. Es wird vorher in Javascript übersetzt, was Typescript mit bestehenden Javascript-Laufzeitumgebungen kompatibel macht. Im Gegensatz zu Javascript können in Typescript Datentypen explizit definiert werden. Dies ermöglicht die Erkennung von Fehlern bereits bei der Entwicklung, nicht erst bevor der Code ausgeführt wird. [14]

Im Rahmen dieser Diplomarbeit bildet Typescript die Grundlage für das im Frontend eingesetzte Angular, sowie das im Backend eingesetzte NestJS. Die gemeinsame Programmiersprache der zwei Frameworks ermöglicht eine klare und konsistente Definition von Modellen, Datenstrukturen und Schnittstellen. Dadurch wird die Entwicklung deutlich erleichtert und mögliche Fehlerquellen werden reduziert. [15]

Typescript erweitert Javascript und fördert eine strukturierte und wartbare Codebasis. Der Einsatz der Programmiersprache in Angular und NestJS fördert die Übersichtlichkeit und Erweiterbarkeit des gesamten Systems und ist damit sinnvoll für die Umsetzung des Projekts. [15]

2.4 Datenschutz und Verifizierung

In webbasierten Systemen, die personenbezogene Daten verarbeiten, sind Datenschutz und Zugriffssicherheit zentrale Qualitäts- und Schutzanforderungen. Für den in dieser Diplomarbeit betrachteten Bestellservice bedeutet dies, dass sowohl die rechtlichen Rahmenbedingungen der DSGVO als auch geeignete technische und organisatorische Maßnahmen konsequent berücksichtigt werden müssen. Neben der datenschutzkonformen Erhebung, Verarbeitung und Speicherung von Kundendaten ist insbesondere sicherzustellen, dass ausschließlich berechtigte Personen Zugriff auf geschützte Ressourcen erhalten. Daher werden im folgenden Abschnitt wesentliche Grundlagen der DSGVO sowie zentrale Sicherheitsmechanismen wie Verifizierung, Authentifizierung, Token-basierte Verfahren und CORS erläutert und in den Kontext moderner Webanwendungen eingeordnet.

2.4.1 Datenschutz und DSGVO

Im Rahmen dieser Diplomarbeit spielt Datenschutz eine wichtige Rolle, weil auch der Bestellservice die Daten der Kunden erfassen muss. Gemäß der Datenschutzgrundverordnung (DSGVO) beschreibt Datenschutz alle Maßnahmen, die den Missbrauch und die unbefugte Weitergabe von personenbezogenen Daten verhindern soll. [16]

Dabei geht es nicht nur um rechtliche, sondern auch um technische und organisatorische Vorgaben, die auch der Bestellservice einhalten muss. Die Rechtliche Grundlage bildet die DSGVO, die seit 2018 in der gesamten EU gültig ist. Sie legt fest, dass Daten nur dann erfasst und verarbeitet werden dürfen, wenn ein klarer Zweck besteht und betroffenen Personen darüber informiert werden. Außerdem sollen die Nutzer stets Kontrolle über die eigenen Daten haben. [16, 17]

Zur Sicherung und Umsetzung dieser gesetzlichen Anforderungen, spielt auch die technische Datensicherung eine wichtige Rolle. Zentral befinden sich hier Themen wie Verschlüsselung, Backups sowie Zugriffsbeschränkungen, mit denen sich beschäftigt werden muss. Aus diesem Grund ist es besonders wichtig schon bei der Entwicklung darauf zu achten. [18]

Bei der Speicherung der Daten, beispielweise mit einem Cloud-Dienst wie Microsoft Azure, muss sichergestellt werden, dass die Daten ausreichend geschützt sind. Daher sind Cloud-Dienste mit Rechenzentren außerhalb der EU mit Vorsicht zu betrachten bzw. möglicherweise zu vermeiden. Als Ausnahme gelten seit 2023 die Vereinigten Staaten von Amerika, mit der 2023 im sog. EU-US Data Privacy Framework vereinbart wurde, wann und wie personenbezogene Daten aus der EU in die USA übertragen werden dürfen. [19]

2.4.2 Verifizierung und Authentifizierung

Neben Datenschutz spielt auch Verifizierung und Authentifizierung eine wichtige Rolle. Dies ist vor allem essentiell, um Kundendaten vor unberechtigtem Zugriff zu schützen. Beide Begriffe werden gerne verwendet, beschreiben aber unterschiedliche Vorgänge. [16, 20]

Unter Verifizierung versteht man die Überprüfung bzw. Legitimierung, ob eine angegebene Identität echt ist. Dies kann beispielsweise bei der Registrierung erfolgen, indem die Echtheit der E-Mail-Adresse bestätigt wird. [16, 20]

Die Authentifizierung beschäftigt sich hingegen mit dem Anmeldevorgang selbst. Dabei wird festgestellt, dass jemand wirklich die Person ist, als die er oder sie sich ausgibt. Dafür gibt es verschiedene Verfahren z.B. die Eingabe von Passwörtern. [20, 21]

In modernen webbasierten Systemen wird die Authentifizierung zunehmend über sogenannte Third-Party-Logins umgesetzt. Dabei haben die User den Vorteil, ein Konto eines externen Anbieters zu verwenden, um sich bei der Anwendung anzumelden. Externe Anbieter für einen solchen Dienst sind z.B. Microsoft oder Google. Neben der Anmeldefunktion liefern diese auch zusätzliche Sicherheitsmechanismen wie die Multi-Faktor-Authentifizierung, womit das Risiko von unbefugtem Zugriff deutlich reduziert werden kann. [20, 22]

In dieser Diplomarbeit wird die Anmeldung über die Schul-E-Mail-Adressen der User erfolgen, wie genau das umgesetzt wird ist im späteren Kapitel [Authentifizierung und E-Mail-Verifikation in Webanwendungen](#) ersichtlich.

Authentifizierung in Webanwendungen

Die Authentifizierung stellt einen zentralen Bestandteil moderner Webanwendungen dar und dient dazu, den Zugriff auf geschützte Ressourcen ausschließlich berechtigten Personen zu ermöglichen. Ziel ist es sicherzustellen, dass ein Benutzer tatsächlich die Identität besitzt, die er vorgibt zu haben. Insbesondere bei Systemen, die personenbezogene Daten verarbeiten, ist eine sichere Authentifizierung unbedingt notwendig, um Datenschutzanforderungen einzuhalten und Missbrauch zu verhindern [16, 21].

Grundsätzlich unterscheidet man zwischen verschiedenen Authentifizierungsfaktoren. Dazu zählen Wissensfaktoren (z. B. Passwörter), Besitzfaktoren (z. B. Tokens oder Mobilgeräte) sowie Inhärenzfaktoren (z. B. biometrische Merkmale). Je mehr dieser Faktoren kombiniert werden, desto höher ist das Sicherheitsniveau des Systems. Dieses Prinzip wird als Mehrfaktorauthentifizierung bezeichnet und gilt als bewährte Methode zur Absicherung von Benutzerkonten. [16, 17, 21]

In webbasierten Anwendungen erfolgt die Authentifizierung häufig zustandslos. Das bedeutet, dass der Server keine dauerhafte Sitzung speichert, sondern der Authentifizierungsstatus über kryptografisch gesicherte Informationen übertragen wird. Eine weit verbreitete Lösung hierfür sind sogenannte Token-basierte Verfahren, die insbesondere bei REST-basierten Architekturen eingesetzt werden [23–25].

Verwendung und Einsatz von JWT

Die Token-basierte Authentifizierung ist ein Verfahren, bei dem Benutzern nach erfolgreicher Anmeldung ein digitales Token ausgestellt wird. Dieses Token wird bei weiteren Anfragen an den Server mitgesendet. Der Server überprüft das Token und entscheidet anhand dessen, ob der Zugriff erlaubt ist [23].

Dazu gibt es viele verschiedene Technologien. In dieser Diplomarbeit wird dabei besonders auf JSON Web Token (JWT) eingegangen. Ein JWT besteht aus drei Teilen: Header, Payload und Signatur. Während der Header Metadaten zum Token enthält, speichert die Payload Informationen über den Benutzer, beispielsweise eine Benutzer-ID oder Rollen. Die Signatur stellt sicher, dass das Token nicht manipuliert wurde und garantiert dessen Integrität. [23, 24] Neben der eigentlichen Authentifizierung können diese Tokens auch zur Verifizierung verwendet werden. Beispielsweise bei der Bestätigung von E-Mail-Adressen oder beim Zurücksetzen von Passwörtern. [21, 26]

Dieses Token-basierte Verfahren reduziert das Risiko unbefugter Zugriffe erheblich, da Tokens nur für einen begrenzten Zeitraum gültig sind und nicht wiederverwendet werden können. [23]

Cross-Origin Resource Sharing (CORS)

Neben der Authentifizierung spielt auch die Zugriffskontrolle auf Netzwerk- bzw. Browser-Ebene eine bedeutende Rolle. Eine wichtige Maßnahme ist dabei Cross-Origin Resource Sharing (CORS). CORS ist ein vom Webbrowser unterstützter Sicherheitsmechanismus, der Zugriffe aus Webanwendungen kontrolliert, sobald Anfragen von einer anderen Origin (z. B. anderer Domain oder Port) kommen. Dadurch lässt sich gezielt festlegen, welche Frontends auf bestimmte Ressourcen der Server-API zugreifen dürfen und welche nicht. [27, 28]

Eine korrekt konfigurierte CORS-Strategie erhöht die Sicherheit wesentlich, weil sie unkontrollierte Cross-Origin-Zugriffe verhindert und damit das Risiko bestimmter Cross-Site-Angriffe reduziert. Gleichzeitig sollte die Konfiguration praxisnah bleiben: Wird sie zu restriktiv gesetzt, können legitime Szenarien (z. B. Test- und Produktionsumgebungen) unnötig blockiert werden und die Nutzbarkeit leidet. Damit stehen Zugriffskontrollen und Sicherheit im Kontrast zur Benutzbarkeit des Systems. [27, 28]

2.4.3 Zusammenfassung

Zusammenfassend zeigt dieses Kapitel, dass Datenschutz und Identitätsprüfung eng miteinander verknüpft sind: Die DSGVO definiert verbindliche Anforderungen an Zweckbindung, Transparenz und Kontrolle über personenbezogene Daten und erfordert zugleich technische Schutzmaßnahmen wie Verschlüsselung, Backups und Zugriffsbeschränkungen. Verifizierung und Authentifizierung adressieren dabei unterschiedliche Sicherheitsziele, nämlich die Bestätigung einer Identität bzw. den sicheren Anmeldevorgang. In Webanwendungen haben sich Token-basierte Ansätze, insbesondere JWT, als verbreitete Lösung etabliert, wobei eine sichere Token-Erzeugung, zeitliche Begrenzung und sorgfältige Behandlung sensibler Inhalte entscheidend sind. Ergänzend unterstützt CORS die Zugriffskontrolle zwischen unterschiedlichen Origins und reduziert das Risiko unautorisierter Anfragen. Insgesamt bilden diese Konzepte die Grundlage für eine datenschutzkonforme und robuste Sicherheitsarchitektur des Systems.

2.5 PostgreSQL als relationale Datenbank

Relationale Datenbanken sind für viele Informationssysteme unverzichtbar, weil sie Daten strukturiert speichern, zuverlässig verarbeiten und effizient abfragen können. Auch für die in dieser Diplomarbeit entwickelte Anwendung ist eine stabile Persistenzschicht wichtig: Bestell- und Kundendaten müssen dauerhaft gespeichert werden und auch bei gleichzeitigen Zugriffen korrekt und konsistent bleiben.

Dieses Kapitel stellt daher zuerst grundlegende Konzepte relationaler Datenbanken vor (z. B. Schlüsselbeziehungen und Normalisierung) und erklärt anschließend die ACID-Eigenschaften als Basis für transaktionssichere Verarbeitung. Darauf aufbauend wird PostgreSQL als konkretes Datenbankmanagementsystem vorgestellt – mit Fokus auf Architektur, Erweiterbarkeit und Sicherheitsmechanismen – sowie der Zugriff darauf in containerisierten Umgebungen betrachtet.

2.5.1 Relationale Datenbanken im Allgemeinen

Relationale Datenbanken bilden die Grundlage von modernen Softwarelösungen. Sie ermöglichen die strukturierte Speicherung, Verwaltung und Abfrage großer Datenmengen. Das relationale Datenmodell wurde 1970 von Edgar F. Codd entwickelt und gilt bis heute als Meilenstein in der Informatik [29]. Es basiert auf mathematischen Konzepten der Mengenlehre und Prädikatenlogik, welche eine Grundlage für die verschiedenen Datenoperationen darstellen [29, 30].

Im relationalen Modell werden Daten in Tabellen (Relationen) dargestellt, die aus Zeilen (Tupeln) und Spalten (Attributen) bestehen. Jede Tabelle beschreibt eine Entität, zum Beispiel „Kunde“ oder „Bestellung“. Eine Zeile stellt einen Datensatz dar, während jede Spalte ein Attribut des Datensatzes definiert, etwa *Name* oder *KundenID*. [30]

Ein zentrales Ziel relationaler Systeme ist die Datenunabhängigkeit. Dabei sind die Daten in logischen Strukturen von der physischen Speicherung getrennt, sodass Änderungen am internen Speicherformat keine Auswirkungen auf Anwendungen haben. Dies ermöglicht Flexibilität und Erweiterbarkeit [30]. Diese Trennung wird auch als „Trennung von logischem und physischem Schema“ bezeichnet. [30]

Die Datenbank organisiert Informationen in einem bestimmten Daten-Schemata, in dem Tabellen, Indizes, Views und Zugriffsrechte definiert sind. Über die Abfragesprache SQL (Structured Query Language) erfolgt der Zugriff auf Daten. SQL ist eine deklarative Sprache, wodurch die Bedienung relativ einfach ist: der User beschreibt einfach die benötigten Daten. [30, 31]

Primär- und Fremdschlüssel:

Jede Tabelle besitzt einen Primärschlüssel, mit dem ein Datensatz eindeutig identifiziert werden kann. Dieser Schlüssel stellt sicher, dass keine Zeilen vollständig ident sind. Der Primärschlüssel kann aus einen oder mehreren Attributen bestehen. Beispielsweise kann die *KundenID* als Primärschlüssel verwendet werden, um damit die Kunden eindeutig zu beschreiben und identifiziert zu können. Der wesentliche Vorteil von Primärschlüsseln ist also die Datenintegrität – kein Datensatz kann doppelt vorkommen. Ohne einen geeigneten Primärschlüssel würde es zu Mehrdeutigkeit kommen [30, 31]. Würden beispielsweise die *Kunden* alleine mit dem Namen identifiziert werden, kann es leicht zu Doppelnennungen und Missverständnissen kommen. [30, 31]

Fremdschlüssel definieren die Beziehungen zwischen Tabellen. Ein Fremdschlüssel einer Tabelle verweist auf den Primärschlüssel einer anderen Tabelle. Auf diese Weise können Daten verschiedener Tabellen logisch miteinander verknüpft werden [30, 31]. Zum Beispiel kann in einer Tabelle *Bestellungen* die Spalte *KundenID* als Fremdschlüssel auf die *KundenID* der Tabelle *Kunde* verweisen.

Daraus ergeben sich verschiedene Beziehungen zwischen den Tabellen, die Multiplizitäten genannt werden. Infolgedessen ergibt sich beispielsweise eine 1:n-Beziehung. Das bedeutet, dass ein Datensatz kann mit mehreren anderen Datensätzen verknüpft ist. [30, 31]

In der Tabelle *Bestellungen* kann ein Kunde daher mehrfach vorkommen, weil er mehrere Bestellungen tätigen kann. Um diese Zuordnung herzustellen, enthält die Tabelle *Bestellung* die Spalte *KundenID* als Fremdschlüssel. Damit wird jede Bestellung eindeutig einem Kunden zugewiesen.

Diese referenzielle Verknüpfung ist eine zentrale Stärke relationaler Datenbanksysteme, da sie Konsistenz zwischen abhängigen Tabellen sicherstellt. Wird beispielsweise ein Kunde gelöscht, kann die Datenbank – abhängig von der definierten Regel (z. B. *Cascade* oder *Restrict*) – entweder automatisch alle zugehörigen Bestellungen entfernen oder den Löschvorgang verhindern. Dadurch werden Dateninkonsistenzen vermieden werden. [30, 31]

Normalisierung

Ein weiteres zentrales Konzept ist die Normalisierung. Ihre Prinzipien bestimmen, wie und auf welche Art und Weise Daten in unterschiedliche Tabellen und Datensätze geschrieben werden können. Ziel ist es Redundanz (doppelte Daten) zu reduzieren und Datenanomalien (fehlerhafte Daten) zu vermeiden. Dies ermöglicht eine logische und stabile Struktur, in der Informationen nur einmal gespeichert werden. [32]

Erste Normalform (1NF)

Eine Tabelle befindet sich in der ersten Normalform (1NF), wenn alle Attribute atomar sind, das bedeutet, dass jedes Feld genau einen unteilbaren Wert enthält. Ein Verstoß liegt vor, wenn in einer Spalte mehrere Informationen zusammengefasst werden. [32]

Ein typisches Beispiel wäre eine Spalte Adresse, in der sowohl Straße als auch Hausnummer gemeinsam gespeichert werden (z. B. „Hauptstraße 12“). Da es sich dabei eigentlich um zwei getrennte Werte handelt (Straße und Hausnummer), sollte dies in zwei eigene Spalten aufgeteilt werden. So bleibt die Struktur eindeutig und Abfragen (z. B. nach allen Kunden in einer bestimmten Straße oder Hausnummer) werden zuverlässiger. [32]

Zweite Normalform (2NF)

Die zweite Normalform verlangt zusätzlich, dass jedes Nichtschlüsselattribut vollständig vom Primärschlüssel abhängt. Befindet sich der Primärschlüssel aus mehreren Spalten (z. B. einer Kombination aus *KundenID* und *ProduktID*), darf kein Attribut nur von einer der beiden Spalten abhängen. Ansonsten wird eine zusätzliche Tabelle erzeugt, um die Abhängigkeit sauber abzubilden. [32]

Wenn beispielsweise in einer Tabelle *Bestellpositionen* mit einem zusammengesetzten Schlüssel (*BestellID*, *ProduktID*) wäre das Attribut *Produktname* problematisch, weil es nur von *ProduktID* abhängt und nicht von der *BestellID*. Der *Produktname* gehört daher in eine eigene Tabelle *Produkte* (*ProduktID*, *Produktname*, ...), während die Tabelle *Bestellpositionen* nur die Verknüpfung mit Primär- bzw. Fremdschlüssel und bestellbezogene Daten enthält. [32]

Dritte Normalform (3NF)

In der dritten Normalform dürfen Nichtschlüsselattribute nur vom Primärschlüssel abhängen, nicht aber voneinander. Angenommen es wird in einer Kundentabelle sowohl *Postleitzahl* als auch *Ort* gespeichert, besteht eine Abhängigkeit zwischen diesen beiden Attributen. Der Ort sollte in einer eigenen Tabelle gespeichert werden, um Dateninkonsistenzen zu vermeiden. [32]

Die Normalisierung führt somit zu einer klaren und logischen Struktur der Datenbank. In der Praxis wird sie oft mit bewusster Denormalisierung kombiniert, um Abfragegeschwindigkeit zu optimieren – ein typischer Zielkonflikt zwischen Datenintegrität und Performance. [32]

ACID-Transaktionen

Transaktionen sind das zentrale Konzept von relationalen Datenbanksystemen. Besonders bei geschäftskritischen Informationen oder Szenarien, z.B. dem gleichzeitigen Zugriff von mehreren Benutzern, ist es entscheidend, dass die Datenbank jederzeit Integrität und Konsistenz gewährleisten kann. Damit Datenoperationen auch unter diesen Bedingungen fehlerfrei ausgeführt werden können, müssen sie einem klar definierten Satz an Eigenschaften entsprechen. [33] Diese Eigenschaften sind als ACID-Properties bekannt und beschreiben die grundlegenden Anforderungen an robuste Transaktionssysteme. [33] ACID ist ein Akronym und steht für A – Atomicity, C – Consistency, I – Isolation, D – Durability.

Atomicity (Atomarität)

Die Atomarität beschreibt die Unteilbarkeit einer Transaktion. Sie garantiert, dass sämtliche in einer Transaktion enthaltenen Operationen entweder vollständig ausgeführt oder im Fehlerfall vollständig rückgängig gemacht werden. Zwischenzustände, in denen nur Teile der Operationen wirksam sind, sind somit ausgeschlossen. [33]

Ein klassisches Beispiel ist der Geldtransfer zwischen zwei Konten: Das Abbuchen von einem Konto und das Gutschreiben auf einem anderen Zielkonto bilden eine logische Einheit. Schlägt einer dieser Schritte fehl, führt das Datenbanksystem ein Rollback durch und stellt den ursprünglichen Zustand wieder her. [33]

Technisch wird die Atomarität durch Mechanismen wie interne Transaktionsprotokolle und Zwischenspeicher umgesetzt, wodurch Änderungen an den Datensätzen erst dann endgültig übernommen werden, wenn ein Commit erfolgreich bestätigt wurde. [33]

Consistency (Konsistenz)

Die Konsistenz fordert, dass jede Transaktion die Datenbank von einem gültigen in einen ebenfalls gültigen Zustand überführt. Eine Transaktion darf keine der im System festgelegten Integritäts-, Referenz- oder Geschäftsregeln verletzen [31, 33]. Zu diesen Regeln gehören unter anderem Schlüsselbeziehungen, zulässige Wertebereiche oder notwendige Geschäftslogiken. Sobald eine Operation innerhalb einer Transaktion gegen eine solche Regel verstößt, bricht das Datenbanksystem die gesamte Transaktion ab und stellt mittels Rollback den konsistenten Ausgangszustand wieder her. Die Konsistenz ist also keine isolierte Eigenschaft, sondern wird im Zusammenspiel mit den übrigen ACID-Eigenschaften betrachtet. [31, 33]

Isolation

Isolation bedeutet, dass parallellaufende Transaktionen sich so verhalten sollen, als würden sie nacheinander ausgeführt. Jede Transaktion arbeitet dabei „für sich“ und darf die andere nicht unbeabsichtigt beeinflussen. Das ist vor allem in Mehrbenutzersystemen wichtig, also überall dort, wo viele Zugriffe gleichzeitig passieren. [31, 33]

Ohne ausreichende Isolation kann es zu typischen Problemen kommen: Es werden Daten gelesen, die noch gar nicht endgültig gespeichert wurden (*dirty reads*), erhält bei derselben Abfrage unterschiedliche Ergebnisse (*non-repeatable reads*) oder es tauchen plötzlich zusätzliche

Datensätze auf, weil eine andere Transaktion parallel neue Einträge erstellt (*phantom reads*). [31, 33]

Um diese Effekte zu kontrollieren, definiert der SQL-Standard verschiedene Isolationsstufen. Sie legen fest, wie stark parallellaufende Transaktionen voneinander abgeschirmt werden sollen. PostgreSQL nutzt dafür zusätzlich Multi-Version Concurrency Control (MVCC): Anstatt, dass sich Transaktionen ständig gegenseitig sperren, arbeitet jede Transaktion mit einem passenden „Datenstand“ bzw. einer eigenen Sicht auf die Daten. [31, 33]

Das sorgt für hohe Parallelität, weil Lese- und Schreibzugriffe sich deutlich seltener blockieren. Dadurch bleibt die gewünschte Isolation auch bei hoher Last erhalten – und gleichzeitig profitieren Effizienz und Skalierbarkeit des Gesamtsystems. [31, 33]

Durability (Dauerhaftigkeit)

Die Dauerhaftigkeit stellt sicher, dass das Ergebnis einer erfolgreichen Transaktion dauerhaft und vollständig gespeichert wird und auch bei Systemausfällen und Hardwarefehlern nicht verloren gehen kann. Erst nach dem Commit gilt eine Transaktion, als endgültig und das Ergebnis wird daraufhin gespeichert. Um dies technisch zu gewährleisten, verwenden relationale Systeme robuste Protokollierungsmechanismen wie Write-Ahead Logging. Dabei werden sämtliche Änderungsinformationen zunächst zuverlässig in ein Transaktionsprotokoll geschrieben, bevor sie in die eigentlichen Datensätze übertragen werden. Im Falle eines Absturzes ermöglicht dieses Log eine präzise Wiederherstellung aller bestätigten Transaktionen, so dass das Datenbanksystem seinen konsistenten Zustand vollständig rekonstruieren kann. [31, 34]

Zusammenwirken der ACID-Eigenschaften

Die vier ACID-Eigenschaften entfalten ihre Bedeutung erst im Zusammenspiel. Während Atomarität und Dauerhaftigkeit primär die physische Integrität eines Systems sichern, indem sie die vollständige Ausführung sowie die dauerhafte Speicherung erfolgreicher Transaktionen garantieren, stellen Konsistenz und Isolation die logische Korrektheit sämtlicher Operationen sicher. Die Datenkonsistenz gewährleistet, dass die Datenbank ausschließlich Zustände annimmt, die sämtlichen strukturellen und fachlichen Regeln entsprechen. Hingegen verhindert die Isolation, dass sich parallel ausgeführte Transaktionen gegenseitig beeinflussen und dadurch widersprüchliche oder fehlerhafte Ergebnisse entstehen. Erst das Zusammenspiel dieser Mechanismen ermöglicht eine Datenverarbeitung, die selbst unter hoher Last z.B. bei umfangreichen Mehrbenutzerzugriffen und trotz möglicher technischer Störungen korrekte Resultate liefert. [31]

Das ACID-Modell bildet damit die unverzichtbare Grundlage für Integrität, Sicherheit und Nachvollziehbarkeit moderner Informationssysteme. Besonders in Bereichen mit hohen Anforderungen an die Verlässlichkeit, beispielsweise dem Finanzsektor oder Gesundheitswesen, ist die konsequente Einhaltung dieser Prinzipien entscheidend für die Qualität, Belastbarkeit und regulatorische Konformität der Datenverarbeitung. [31]

2.5.2 PostgreSQL als DBMS

PostgreSQL ist ein objektrelationales Datenbankmanagementsystem (ORDBMS), dessen Ursprünge sich in einem Projekt der University of California in Berkeley unter der Leitung von Michael Stonebraker finden lässt. Ziel dieses Projektes (mit dem Namen POSTGES) war, das klassische relationale Modell um Konzepte wie Objekttypen, Vererbung und erweiterbare Operatoren zu erweitern [35]. Im Jahr 1996 wurde aus dem Projektrahmen heraus PostgreSQL als Open-Source Projekt geschaffen. Diese Freigabe sorgt für eine kontinuierliche, community-getriebene Entwicklung, die durch die PostgreSQL Global Development Group organisiert wird. Auf diese Weise wurde PostgreSQL zu einer der führenden Open-Source-Datenbanksystemen und wird bis heute sowohl in wissenschaftlichen als auch in industriellen und behördlichen Kontexten verbreitet eingesetzt. [35, 36]

Architektur und Systemaufbau

Architektonisch folgt PostgreSQL einem Client-Server-Modell, bei dem ein zentraler Serverprozess alle eingehenden Verbindungen verwaltet. Pro Client werden dabei separate Prozesse bzw. Backend-Einheiten verwendet, wodurch Sitzungen voneinander isoliert werden und Fehler nicht zwangsläufig das Gesamtsystem beeinträchtigen. Der interne Aufbau ist modular, wodurch Verarbeitungsschritte wie Parsing, Optimierung und Ausführung getrennt erfolgen können und die Abfrageverarbeitung systematisch strukturiert ist. [36, 37]

Für die Parallelität und Konsistenz bei gleichzeitigen Zugriffen setzt PostgreSQL auf MVCC-basierte Verfahren. Dadurch können Lese- und Schreibzugriffe besser gleichzeitig ausgeführt werden, ohne mit Sperren arbeiten zu müssen. [37]

2.5.3 Zugriff auf PostgreSQL in containerisierten Umgebungen

Mit der Verbreitung containerbasierter Architekturen (siehe [Kapitel 2.6](#)) hat sich auch der Betrieb von Datenbanksystemen verändert. PostgreSQL lässt sich aufgrund seiner stabilen Client-Server-Architektur in solchen Umgebungen typischerweise in einem eigenen Container betreiben, während Anwendungsdienste in separaten Containern laufen. [31, 33, 36, 38]

Die Kommunikation zwischen Anwendung und Datenbank erfolgt auch im Container-Kontext über standardisierte Netzwerkprotokolle, insbesondere über TCP/IP. Containerisierte Dienste können dabei über interne virtuelle Netzwerke miteinander verbunden werden. Innerhalb eines solchen Netzwerks ist der Datenbankcontainer über einen definierten Hostnamen erreichbar, der in der Regel dem Containernamen oder einem konfigurierten Servicenamen entspricht. Auf diese Weise wird eine lose Kopplung zwischen Anwendung und Datenbank erreicht, was die Wartbarkeit und Skalierbarkeit des Systems erhöht. [39, 40]

Für den Zugriff auf PostgreSQL werden in containerisierten Szenarien dieselben Mechanismen verwendet wie in klassischen Installationen. Dazu zählen die Angabe von Verbindungsparametern wie Host, Port, Datenbankname sowie Benutzer- und Authentifizierungsinformationen. Diese Parameter werden häufig über Umgebungsvariablen bereitgestellt, um Konfigurationsdaten vom eigentlichen Anwendungscode zu trennen. [38, 40, 41]

Ein weiterer wesentlicher Aspekt beim Einsatz von PostgreSQL in Containern ist die Persistenz der Daten. Da Container selbst als kurzlebig gelten, wird der eigentliche Datenbestand in sogenannten Volumes oder externen Speichermechanismen abgelegt. Dadurch bleibt der Datenbestand auch bei einem Neustart oder Austausch des Containers erhalten und entspricht den Anforderungen an Datenintegrität und Verfügbarkeit. [39, 41]

Aus sicherheitstechnischer Sicht gelten auch im Container-Betrieb die etablierten Schutzmechanismen von PostgreSQL. Zugriffsbeschränkungen, rollenbasierte Berechtigungen sowie verschlüsselte Kommunikationskanäle können unverändert eingesetzt werden. Zusätzlich ermöglicht die Netzwerkisolation von Containern eine weitere Reduktion der Angriffsfläche, da Datenbankdienste ausschließlich für autorisierte Anwendungscontainer erreichbar gemacht werden können. [38, 39]

Insgesamt stellt der containerisierte Betrieb von PostgreSQL eine flexible und robuste Möglichkeit dar, relationale Datenbanken in moderne Anwendungssysteme zu integrieren. Die Trennung von Anwendung und Datenbank, kombiniert mit standardisierten Zugriffsschnittstellen, unterstützt sowohl eine saubere Systemarchitektur als auch eine vereinfachte Verwaltung im Entwicklungs- und Produktionsbetrieb. [38–41]

2.5.4 Zusammenfassung

Zusammenfassend verdeutlicht dieses Kapitel, warum relationale Datenbanken eine zentrale Grundlage für eine konsistente und nachvollziehbare Datenhaltung sind. Primär- und Fremdschlüssel sowie die Normalisierung unterstützen die Datenintegrität, indem sie Beziehungen eindeutig abbilden und Redundanz bzw. unnötige Doppelungen reduzieren. ACID-Transaktionen stellen zudem sicher, dass Änderungen auch bei parallelen Zugriffen und in Fehlerfällen korrekt und verlässlich verarbeitet werden.

PostgreSQL bietet dafür als objektrelationales und erweiterbares DBMS eine robuste Architektur: Es unterstützt eine breite Palette an Datentypen, lässt sich flexibel erweitern und stellt differenzierte Mechanismen für Sicherheits- und Zugriffskontrollen bereit. Im containerisierten Betrieb kann PostgreSQL nahtlos in moderne Systemarchitekturen integriert werden. Besonders wichtig sind dabei Netzwerkisolation, ausgelagerte Konfiguration und persistente Volumes, da sie zentrale Anforderungen an Wartbarkeit, Verfügbarkeit und Datensicherheit zuverlässig abdecken.

2.6 Grundlagen zur Microservice-Architektur

Die Microservice-Architektur ist ein zentrales Konzept moderner Softwareentwicklung. Im Gegensatz zu monolithischen Systemen wird eine Anwendung hierbei in mehrere unabhängige Services zerlegt. Jeder Service übernimmt eine klar definierte Aufgabe und wird in einem eigenen Prozess betrieben. Entwicklung, Deployment und Skalierung erfolgen unabhängig voneinander, was Flexibilität und Wartbarkeit erhöht. [42–44]

2.6.1 Grundlagen und wesentliche Merkmale

Die Grundidee der Microservice-Architektur besteht darin, komplexe Anwendungen in kleine, eigenständige Dienste aufzuteilen. Dadurch lassen sich Verantwortlichkeiten klar voneinander abgrenzen und einzelne Komponenten gezielt weiterentwickeln oder skalieren. Im Vergleich zu monolithischen Systemen führt dies zu höherer Entwicklungs- und Betriebseffizienz sowie zu einer besseren Nutzung von Rechenressourcen [42–44].

Zentrale Prinzipien der Microservice-Architektur sind:

- **Lose Kopplung:** Services sind möglichst unabhängig und kommunizieren nur über klar definierte Schnittstellen (z. B. APIs). Interne Änderungen bleiben dadurch lokal, solange die Schnittstelle stabil bleibt [42–44].
- **Hohe Kohäsion:** Ein Service bündelt fachlich zusammengehörige Funktionen und verfolgt einen klaren Zweck. Das erhöht Verständlichkeit und Wartbarkeit. [42–44]
- **Unabhängiges Deployment:** Services können separat entwickelt, getestet und ausgerollt werden. Änderungen an einem Service sollen nicht automatisch Releases anderer Services erzwingen. [42–44]
- **Verteilte Teamverantwortung:** Teams werden entlang von fachlichen oder technischen Servicegrenzen organisiert und übernehmen Ownership für „ihre“ Services (Entwicklung und Betrieb). [42–44]
- **Datenhoheit:** Jeder Service verwaltet seine eigenen Daten (Modell und Speicherung). Andere Services greifen nicht direkt auf diese Datenbank zu, sondern über Schnittstellen oder Events. [42–44]
- **Single-Responsibility-Prinzip:** Jeder Service hat genau eine Hauptverantwortung. Dadurch bleiben Umfang und Komplexität beherrschbar. [42–44]

2.6.2 Kommunikation und Koordination von Micro-services

Die Zerlegung in viele Services erfordert klar definierte Kommunikationsmechanismen. Grundsätzlich wird zwischen synchroner und asynchroner Kommunikation unterschieden.

Bei der **synchronen Kommunikation** ruft ein Service einen anderen direkt über REST-APIs oder vergleichbare Schnittstellen auf und wartet auf dessen Antwort. Dies ist einfach umzusetzen, kann jedoch bei Ausfällen oder Latenzen zu Blockierungen im Gesamtsystem führen. [42, 43]

Die **asynchrone Kommunikation** nutzt Message Broker wie RabbitMQ oder Apache Kafka. Nachrichten werden in Queues abgelegt und unabhängig verarbeitet, was eine stärkere Entkopplung, bessere Lastverteilung und höhere Fehlertoleranz ermöglichen. Gleichzeitig steigt der Aufwand für Konsistenzsicherung und Monitoring. [42, 43]

Zusätzlich wird zwischen **direkter Kommunikation**, bei der Services ihre Zieladressen kennen, und **indirekter Kommunikation** über Broker oder Vermittlungsschichten unterschieden. Letztere erhöht Resilienz und Skalierbarkeit. [42, 43]

2.6.3 Koordination in Microservice-Landschaften

Für die Umsetzung komplexer Geschäftsprozesse müssen mehrere Services koordiniert werden. Zwei grundlegende Ansätze haben sich etabliert.

Bei der **Orchestrierung** steuert eine zentrale Komponente den Prozess: Sie ruft Services in der richtigen Reihenfolge auf, sammelt Ergebnisse und entscheidet über die nächsten Schritte. Das macht Abläufe meist gut nachvollziehbar und erleichtert Monitoring und Tests. Nachteil ist die stärkere Abhängigkeit von dieser Instanz, wodurch ein möglicher zentraler Ausfall- oder Engpasspunkt entsteht. [45]

Bei der **Choreographie** gibt es keine zentrale Steuerung. Services reagieren selbstständig auf **Events** anderer Services und lösen bei Bedarf weitere Events aus. Dadurch ist das System stärker entkoppelt und oft besser skalierbar. Gleichzeitig verteilt sich die Prozesslogik, was Nachvollziehbarkeit, Debugging und Fehleranalyse erschweren kann. [45]

2.6.4 Datenmanagement in Microservices

Ein zentrales Prinzip der Microservice-Architektur ist die Datenhoheit pro Service: Jeder Service verwaltet sein eigenes Datenmodell und idealerweise auch seine eigene Datenbank. Dadurch sinkt die Kopplung zwischen Services, Releases werden unabhängiger und Änderungen an Datenstrukturen bleiben auf einen begrenzten Verantwortungsbereich beschränkt. Gleichzeitig entsteht jedoch die Herausforderung, fachliche Konsistenz über mehrere Services hinweg sicherzustellen. [43, 44, 46]

Klassische, serviceübergreifende **ACID-Transaktionen** sind in verteilten Microservice-Landschaften meist nur mit hohem Koordinationsaufwand und starker Kopplung umsetzbar. Deshalb setzen viele Systeme auf **eventuelle Konsistenz**: Ein global „stimmiger“ Zustand wird nicht sofort, sondern über eine Folge von Schritten und Ausgleichen erreicht. [43, 47]

Zur Umsetzung werden etablierte Muster genutzt. Häufig werden CQRS und Event Sourcing eingesetzt, um Schreib- und Lesezugriffe zu trennen bzw. Zustandsänderungen als eine Folge von Ereignissen nachvollziehbar zu machen. CQRS und Event Sourcing sind Architekturprinzipien, bei denen Schreibvorgänge Änderungen als Ereignisse (Events) erfassen und in einem eigenen Modell speichern, während Lesezugriffe über ein davon getrenntes, optimiertes Read-Model erfolgen, um Performance und Skalierbarkeit zu verbessern. Dadurch wird die Skalierung und die Integrationslogik erleichtert, sie erhöht jedoch den Aufwand für Modellierung, Monitoring und Fehlersuche. [43, 47, 48]

In Cloud-basierten SaaS-Umgebungen sind zwei Datenhaltungsmodelle verbreitet: Beim **Multi-Tenant-Modell** teilen sich mehrere Mandanten eine Datenbank (oder ein Schema), was Ressourcen spart, aber strikte Isolation, sauberes Berechtigungsdesign und schemafähige Erweiterbarkeit erfordert. Beim **Multi-Instance-Modell** werden getrennte Instanzen pro Mandanten betrieben, was Isolation und Anpassbarkeit verbessert, jedoch den Betriebsaufwand erhöht. Containerisierung (z. B. mit Docker) kann diesen Mehraufwand reduzieren, indem Datenbank- und Service-Instanzen standardisiert bereitgestellt und automatisierter betrieben werden. [38, 41, 43, 48]

2.6.5 Kommunikation zwischen Frontend und Backend

In Microservice-basierten Webanwendungen kommuniziert das Frontend mit den Backend-Services über klar definierte Schnittstellen. Das Frontend übernimmt dabei vor allem Darstellung und Benutzereingaben, während im Backend Geschäftslogik, Authentifizierung und Datenpersistenz verarbeitet werden. Diese Aufgabentrennung ist ein zentrales Architekturprinzip und unterstützt Wartbarkeit sowie Skalierbarkeit. [46]

Technisch erfolgt die Kommunikation typischerweise nach dem Client-Server-Prinzip über HTTP/HTTPS. In vielen Systemen wird dabei ein REST-orientierter Ansatz verwendet, bei dem Daten und Funktionen als Ressourcen modelliert und über standardisierte HTTP-Methoden (z. B. GET, POST, PUT, DELETE) abgerufen oder verändert werden. Als Austauschformat kommt meist **JSON** zum Einsatz, da es kompakt, plattformunabhängig und für Maschinen gut verarbeitbar ist. [23, 24, 27, 28, 49]

In Microservice-Architekturen stellt das Backend oft eine API-Schicht bereit, die als zentraler Zugriffspunkt für das Frontend dient. Das Frontend bleibt dadurch von internen Service-Strukturen entkoppelt und nutzt klar definierte Endpunkte. Das erleichtert unterschiedliche Clients (z. B. Web und Mobile) und ermöglicht eine unabhängige Weiterentwicklung der Services. [43, 46]

Wesentlich ist außerdem die Absicherung der Schnittstellen. Die Token-basierte Authentifizierung stellt sicher, dass nur autorisierte Nutzer auf geschützte Ressourcen zugreifen können während zusätzlicher CORS-Regeln, Zugriffe begrenzen, um unautorisierte Anfragen zu verhindern. [23, 24, 27, 28, 49]

2.6.6 Zusammenfassung

Zusammenfassend ist die Microservice-Architektur eine Verbindung von kleinem, autonomem Service mit klaren Verantwortlichkeiten, loser Kopplung und unabhängigem Deployment. Für die Zusammenarbeit der Services sind geeignete Kommunikationsformen entscheidend, wobei synchrone Aufrufe eine einfache Implementierung ermöglichen, während asynchrone Messaging-Ansätze eine stärkere Entkopplung und höhere Fehlertoleranz unterstützen. [11] Zur Koordination komplexer Abläufe kommen Orchestrierung oder Choreographie zum Einsatz, die jeweils unterschiedliche Vor- und Nachteile hinsichtlich Transparenz und Resilienz aufweisen. Im Datenmanagement führt die serviceeigene Datenhoheit zu Herausforderungen bei der Konsistenz, weshalb in der Praxis häufig auf Konzepte wie eventual Consistency zurückgegriffen wird. Die Frontend-Backend-Kommunikation erfolgt typischerweise über standardisierte HTTP(S)-Schnittstellen (häufig REST/JSON) und wird durch Token-basierte Authentifizierung sowie CORS-Richtlinien abgesichert, wodurch eine robuste und skalierbare Gesamtarchitektur unterstützt wird.

2.7 Containerisierung und Docker

Die in [Abschnitt 2.6](#) beschriebenen Vorteile der Microservice-Architektur lassen sich in der Praxis nur dann gut umsetzen, wenn auch die Infrastruktur mitspielt. Containerisierung ist dafür eine zentrale Basistechnologie, weil sie Isolation, Portabilität und Skalierbarkeit effizient unterstützt. Dadurch können mehrere Dienste getrennt voneinander laufen, ohne dass dafür jedes Mal eine komplette virtuelle Maschine benötigt wird – was Ressourcen spart und gleichzeitig flexibel bleibt. [38, 39]

Container sind dabei leichtgewichtige Laufzeitumgebungen. Im Gegensatz zu virtuellen Maschinen emulieren sie kein vollständiges Betriebssystem, sondern nutzen Funktionen des Host-Systems wie Namespaces und Control Groups (cgroups). Damit lassen sich Prozesse voneinander abschirmen und gezielt mit Ressourcenlimits versehen, obwohl sie denselben Kernel verwenden. Diese Architektur ermöglicht es, Services schnell zu starten, einfach zu replizieren und damit besonders gut in verteilten Microservice-Systemen einzusetzen. [38, 39]

Ein wesentlicher Vorteil ist außerdem, dass Container alle Abhängigkeiten eines Dienstes mitbringen. Dadurch werden Unterschiede zwischen Entwicklungs-, Test- und Produktionsumgebung deutlich reduziert. Gleichzeitig profitieren Microservice-Architekturen von den kurzen Startzeiten, weil sich Dienste bei Bedarf schnell horizontal skalieren lassen. [38, 39]

2.7.1 Docker als Technologie der Containerisierung

Docker hat sich als verbreiteter Standard für Containerisierung etabliert und prägt die praktische Umsetzung vieler Microservice-Architekturen. Anwendungen werden zusammen mit ihren Bibliotheken und Laufzeitabhängigkeiten in Container verpackt, sodass sie in einer konsistenten Umgebung ausgeführt werden können. Die dafür notwendige Isolation und Ressourcensteuerung basiert – wie bei Containern allgemein – auf Betriebssystemmechanismen wie Namespaces und Control Groups. Dadurch bleibt die Laufzeitumgebung unabhängig vom jeweiligen Host-System weitgehend gleich. [38, 39]

Zentrales Element des Docker-Ökosystems ist die Docker Engine. Der Docker Daemon (*dockerd*) verwaltet die Erstellung, Ausführung und Überwachung von Containern und stellt hierfür eine REST-basierte Schnittstelle bereit. Die Trennung zwischen Client und Daemon ermöglicht sowohl lokale als auch remote gesteuerte Deployments und bildet eine wichtige Grundlage für automatisierte Microservice-Umgebungen in Cloud-Infrastrukturen. [38, 39]

Docker Images fungieren als grundlegende Verteilungseinheiten. Sie kapseln Anwendungen einschließlich aller Laufzeitabhängigkeiten und basieren auf einem schichtweisen Aufbau mittels Union Filesystems. Unveränderte Schichten können wiederverwendet werden, wodurch Speicherbedarf, Netzwerktransfer und Update-Zeiten reduziert werden. Die Definition eines Images erfolgt deklarativ in einem Dockerfile, was reproduzierbare Builds und eine hohe Nachvollziehbarkeit sicherstellt. [38, 39]

Docker Container stellen die laufenden Instanzen dieser Images dar. Sie nutzen den Kernel des Hosts, bleiben jedoch hinsichtlich Prozesse, Netzwerk und Dateisystem isoliert. Durch die Begrenzung von Ressourcen mittels cgroups entstehen kurze Startzeiten und geringe Overheads, was eine dynamische Skalierung von Microservices ermöglicht und elastische Cloud-Architekturen unterstützt. [38, 39]

Docker bietet zudem verschiedene Netzwerk- und Speichermechanismen. Für verteilte Microservice-Systeme sind insbesondere Overlay-Netzwerke relevant, da sie eine transparente Kommunikation über mehrere Hosts hinweg ermöglichen. Persistente Daten werden außerhalb der Container über **Volumes** oder externe Speichersysteme verwaltet, was dem Prinzip zustandsloser Services entspricht. [38–41]

Die Verteilung und Versionierung von Images erfolgt über Docker-Registries wie Docker Hub oder cloudbasierte Dienste. Sie stellen eine zentrale Infrastrukturkomponente dar, da Microservice-Architekturen typischerweise eine Vielzahl versionierter Services umfassen. [38, 39]

2.7.2 Orchestrierung von containerisierten Microservices

Mit zunehmender Anzahl containerisierter Services steigt der Aufwand deutlich: Neben dem Starten und Verteilen von Instanzen müssen deren Zustand und Verfügbarkeit kontinuierlich überwacht werden. Treten Fehler auf, müssen Instanzen automatisiert ersetzt werden („re-scheduling“), und bei Lastspitzen ist eine horizontale Skalierung erforderlich, damit Service-Level (z. B. Antwortzeiten) stabil bleiben. Orchestrierungsplattformen wie Kubernetes oder Docker Swarm reduzieren diese Komplexität, indem sie nicht einzelne Container manuell steuern, sondern einen gewünschten Zielzustand beschreiben (z. B. „es sollen dauerhaft drei Instanzen laufen“). Controller- bzw. Managementprozesse gleichen den Ist-Zustand fortlaufend mit diesem Zielzustand ab und führen bei Abweichungen automatisch Gegenmaßnahmen aus. Dies wird dann Self-Healing (Neustart/Neuplatzierung ausgefallener Instanzen) oder Auto-Scaling durch Anpassung der Anzahl der Instanzen bezeichnet. Damit wird der Betrieb in Microservice-Landschaften systematischer, reproduzierbarer und weniger fehleranfällig. [42, 44, 50, 51]

Darüber hinaus übernehmen Orchestrierungssysteme zentrale Querschnittsfunktionen für die Kommunikation in dynamischen Umgebungen: Über Service-Abstraktionen und integrierte Netzwerkmechanismen bleiben Dienste trotz wechselnder Container-IPs erreichbar. Dazu zählen insbesondere Service-Discovery (stabile Service-Namen statt direkter IPs), Netzwerk-/Routing-Konfiguration innerhalb des Clusters sowie Load Balancing zur Verteilung von Anfragen auf mehrere Instanzen. Diese Mechanismen sind essenziell, damit die lose Kopplung von Microservices auch im laufenden Betrieb zuverlässig funktioniert. [40, 44, 50, 51]

Für den produktiven Einsatz sind außerdem kontrollierte Deployment-Strategien wichtig, um Änderungen ohne längere Ausfälle auszurollen. Rolling Updates ersetzen Instanzen schrittweise, sodass der Dienst während des Updates weiter verfügbar bleibt, bei Problemen kann auf eine stabile Version zurückgerollt werden. Blue-Green-Deployments verfolgen ein ähnliches Ziel, indem zwei Umgebungen parallel betrieben und der Traffic kontrolliert umgeschaltet wird, was besonders klare Rückfalloptionen ermöglicht. Insgesamt unterstützen solche Verfahren eine risikoärmere Weiterentwicklung, weil Releases als planbare, überprüfbare Betriebsprozesse umgesetzt werden. [44, 48]

2.7.3 Herausforderungen und Grenzen containerisierter Microservices

Trotz der Vorteile erhöhen Microservices und Containerisierung die Komplexität eines Systems. Insbesondere **Observability** wird zur Schlüsselanforderung: Monitoring, zentrales Logging und verteiltes Tracing müssen systemübergreifend organisiert werden, um Fehlerursachen in verteilten Aufrufketten nachvollziehen zu können. Ohne diese Transparenz steigen Diagnosezeiten und das Risiko von längeren Ausfällen deutlich. [38, 43, 44]

Auch der Aspekt **Sicherheit** bleibt ein zentrales Thema. Container isolieren Prozesse, teilen sich aber den Kernel des Hosts, wodurch Kernel-Schwachstellen auftreten können, die möglicherweise mehrere bzw. alle darauf laufenden Container betreffen können. Daher sind Maßnahmen und ein restriktives Berechtigungsmodell (z. B. Least-Privilege) besonders wichtig. [38, 52]

Schließlich ist **Datenmanagement** ein dauerhafter Komplexitätstreiber: Microservices werden idealerweise zustandslos betrieben, weshalb persistente Daten außerhalb der Container verwaltet werden müssen (z. B. über Volumes). Gleichzeitig erfordert die verteilte Verarbeitung häufig Konsistenzmechanismen jenseits klassischer ACID-Transaktionen, etwa durch eventuelle Konsistenz und kompensierende Abläufe. [41, 42, 45, 53]

2.7.4 Zusammenfassung

Zusammenfassend ist Containerisierung eine zentrale Grundlage für Microservice-Architekturen, da sie Isolation, Portabilität und einen skalierbaren Betrieb bei geringerem Ressourcenverbrauch ermöglichen. Docker unterstützt dies durch ein etabliertes Ökosystem aus Engine, Images und Containern, wodurch Deployments reproduzierbar und das Kapseln von Abhängigkeiten erleichtert wird. In Umgebungen mit vielen Services sind Orchestrierungsplattformen wie Kubernetes oder Docker Swarm wichtig, da sie Skalierung, Service-Discovery und kontrollierte Deployments automatisieren. Gleichzeitig steigen die Anforderungen an Observability (Monitoring, Logging, Tracing) und Security (z. B. Least-Privilege, Image-Scanning, Umgang mit Kernel-Risiken). Insgesamt führen dadurch persistente Daten zu einer höheren Komplexität, weil zustandslose Services externe Speicher- und Konsistenzmechanismen benötigen.

2.8 Cloud Computing und Cloud-Plattformen

Die in [Abschnitt 2.6 \(Microservices\)](#) und [2.7 \(Containerisierung\)](#) beschriebenen Konzepte zeigen ihre Vorteile besonders dann, wenn sie auf Cloud-Plattformen betrieben werden. Cloud-Infrastrukturen stellen Rechenleistung, Speicher und Netzwerk als abstrahierte Ressourcen bereit, die bedarfsgerecht zugewiesen und bei Laständerungen elastisch skaliert werden können. Dadurch eignen sie sich als Grundlage für den stabilen Betrieb verteilter, containerisierter Anwendungen, wie sie in Microservice-Systemen typischerweise entstehen. [54]

Im Folgenden werden zunächst zentrale Begriffe und Eigenschaften des Cloud Computing erläutert. Anschließend werden Amazon Web Services (AWS) und Microsoft Azure als Beispiele für Public-Cloud-Plattformen konzeptionell eingeordnet, ohne konkrete Implementierungsdetails zu vertiefen.

2.8.1 Grundlagen des Cloud Computing

Cloud Computing bezeichnet die Bereitstellung von IT-Ressourcen und IT-Diensten über das Internet nach dem On-Demand-Prinzip. Dazu zählen grundlegende Infrastrukturkomponenten (z. B. Rechenleistung, Speicher und Netzwerke) ebenso wie höherwertige Plattform- und Anwendungsdienste. Charakteristisch sind eine flexible, bedarfsgerechte Nutzung sowie eine nutzungsabhängige Abrechnung, wodurch der Betrieb eigener Hardware in vielen Szenarien entfallen kann. [48, 54]

Cloud-Systeme sind großskalierte, virtualisierte Rechenzentren, in denen zahlreiche Instanzen auf gemeinsamer physischer Infrastruktur betrieben werden [54]. Eine wesentliche technische Grundlage ist die Virtualisierung: Ein Hypervisor abstrahiert physische Ressourcen und weist diese isolierten Instanzen zu, die aus Sicht der Anwender wie eigenständige Server mit eigenem Betriebssystem und Software-Stack erscheinen. [48, 54]

Für den Einsatz in verteilten Architekturen sind insbesondere folgende Eigenschaften wesentlich:

- **Elastizität:** Ressourcen werden automatisch je nach Last hoch- oder heruntergefahren, sodass Leistungsspitzen abgefedert werden. [48, 54]
- **Skalierbarkeit:** Erweiterung bei Bedarf durch mehr Instanzen (Scale-out) oder stärkere Hardware (Scale-up). [48, 54]
- **Resilienz:** Höhere Ausfallsicherheit durch Redundanz und geografische Verteilung, damit ein Ausfall nicht den gesamten Dienst stoppt. [48, 54]
- **Messbarkeit:** Verbrauch von Rechenleistung, Speicher und Netzwerk wird transparent erfasst – Grundlage für Monitoring und nutzungsbasierte Abrechnung. [48, 54]

Diese Eigenschaften sind für Microservice-Architekturen besonders relevant, da einzelne Services häufig unterschiedliche Lastprofile aufweisen und daher unabhängig voneinander skaliert und betrieben werden sollen. [54, 55]

Cloud-Angebote werden meist über die Servicemodelle **Infrastructure as a Service (IaaS)**, **Platform as a Service (PaaS)** und **Software as a Service (SaaS)** beschrieben. Der zentrale Unterschied ist weniger „welche Technik“, sondern **wer welche Schicht betreibt und verantwortet** – also wo die operative Arbeit (Patches, Skalieren, Absichern, Monitoring) liegt. [54]

IaaS (Infrastructure as a Service)

Bei IaaS stellt der Anbieter vor allem die „Roh-Infrastruktur“ bereit: virtuelle Server, Netzwerk und Speicher. Das gibt viel Freiheit, weil man Betriebssystem, Laufzeitumgebung und Konfiguration selbst bestimmen kann. Gleichzeitig bleibt der Großteil der Arbeit beim Betreiber: Updates/Patches, Security-Härtung, Monitoring und der stabile Betrieb der Anwendung müssen selbst organisiert werden. IaaS ist daher besonders geeignet, wenn man viel Kontrolle über Details braucht, verursacht aber vergleichsweise hohen Betriebsaufwand. [54]

PaaS (Platform as a Service)

PaaS nimmt deutlich mehr Arbeitsaufwand ab. Zusätzlich zur Infrastruktur liefert die Plattform typische Betriebsfunktionen wie Deployment, Skalierung, Health-Checks und oft auch eine gemanagte Laufzeitumgebung. Für Teams bedeutet das: Man konzentriert sich stärker auf die Anwendung (Code oder Container), während die Plattform den Standardbetrieb übernimmt. Dadurch wird im laufenden Betrieb der Wartungsaufwand reduziert, jedoch muss sich an gewisse Vorgaben und Einschränkungen der Plattform (z. B. unterstützte Runtimes, Limits, Konfigurationsmodelle) gehalten werden, wodurch die Flexibilität eingeschränkt wird. [54]

SaaS (Software as a Service)

Bei SaaS erhält man eine fertige Anwendung als Dienst. Betrieb, Updates, Skalierung und Sicherheitsbetrieb liegen weitgehend beim Anbieter. Nutzer können meist nur konfigurieren (z. B. Rollen, Benutzer, Einstellungen) und greifen über Weboberflächen oder APIs zu. Das reduziert den technischen Aufwand stark, dafür hat man weniger Einfluss auf Details oder individuelle Anpassungen. [54]

Gerade im Kontext von Microservices ist diese Einordnung hilfreich, weil Services je nach Lastprofil und Betriebsanforderungen unterschiedlich gut zu IaaS, PaaS oder SaaS passen. Unabhängig vom Modell bilden die Cloud-Eigenschaften Elastizität, Skalierbarkeit, Resilienz und Messbarkeit den gemeinsamen Rahmen. [48, 54]

2.8.2 Amazon Web Services vs. Microsoft Azure als Cloud-Anbieter

AWS und Microsoft Azure gehören zu den wichtigsten **Public-Cloud-Plattformen** und werden häufig als Grundlage für den Betrieb verteilter und containerisierter Anwendungen genutzt. Charakteristisch ist, dass Rechenleistung, Speicher und Netzwerk **als standardisierte Dienste** bereitgestellt und bei Bedarf flexibel skaliert werden können. Damit unterstützen solche Plattformen zentrale Cloud-Eigenschaften wie bedarfsgerechte Ressourcenzuteilung und messbare Nutzung. [54]

Ein wichtiger Orientierungspunkt für eine „saubere“ Cloud-Architektur sind die **Well-Architected Frameworks**. Darunter versteht man strukturierte **Bewertungskataloge bzw. Leitfäden**, mit denen Cloud-Architekturen anhand definierter Prinzipien und Fragen systematisch überprüft und verbessert werden können. AWS und Azure bündeln darin Best Practices, um Architekturen nach Qualitätszielen wie Zuverlässigkeit, Sicherheit, Performance-Effizienz, Kostenoptimierung und Betriebsführung zu beurteilen. Diese Leitlinien dienen vor allem als strukturierter Maßstab für Architekturentscheidungen – nicht als konkrete Anleitung für einzelne Dienste. [56–59]

Für Microservice-Architekturen ist zudem relevant, dass solche Plattformen typische Betriebsbausteine bereitstellen, etwa Mechanismen für Skalierung, Überwachung sowie die Einbindung verwalteter Dienste. Dadurch wird der Schritt vom „Server selbst betreiben“ hin zu stärker serviceorientierten Betriebsformen erleichtert. [44, 57, 60]

2.8.3 Cloud-Hosting-Alternativen

Public-Cloud-Plattformen bieten ein sehr breites Spektrum an Diensten, gleichzeitig können Kosten- und Komplexitätsaspekte – abhängig von Umfang und Nutzungsprofil – bei kleineren Projekten rasch spürbar werden. Das hängt u. a. mit dem Prinzip der nutzungsbasierten Bereitstellung und Messbarkeit zusammen, das Cloud-Modelle grundsätzlich prägt. [54] Vor diesem Hintergrund werden im Folgenden zwei Alternativen betrachtet: ein klassischer **vServer/VPS** bei einem nationalen Anbieter (World4You) sowie eine entwicklerorientierte, **PaaS-nahe Plattform** (Render). Ziel ist eine Einordnung hinsichtlich Kontrolle, Betriebsaufwand und Kostenrahmen. [7, 61]

2.8.4 World4You als nationaler Hosting-Anbieter

World4You ist ein österreichischer Hosting-Provider mit Rechenzentren in Österreich und einem Angebot von Webhosting bis zu vServer/VPS-Lösungen. [7, 8] Für containerbasierte Setups sind insbesondere Linux-vServer relevant, weil sie typischerweise die nötige administrative Kontrolle bieten, um eine Container-Runtime wie Docker zu betreiben. [7, 8, 39]

Praktisch bedeutet das: Die Architektur ist weitgehend frei gestaltbar (z. B. getrennte Container für Backend, Datenbank und weitere Dienste). Gleichzeitig liegt der laufende Betrieb stärker in der eigenen Verantwortung. Insbesondere Updates, Backups sowie Monitoring und Logging müssen aktiv geplant und umgesetzt werden, damit Sicherheit und Verfügbarkeit im Produktivbetrieb gewährleistet bleiben. [7, 8, 52]

2.8.5 Render als Cloud-Plattform

Render verfolgt ein PaaS-nahes Modell mit Fokus auf Developer Experience. Anstatt virtuelle Maschinen direkt zu verwalten, werden Services über vordefinierte Service-Typen bereitgestellt, Deployments können Docker-basiert aus Repositories erfolgen und lassen sich gut in Git-Workflows integrieren. [61, 62]

Für datenbankgestützte Anwendungen ist außerdem relevant, dass Render **Managed PostgreSQL** anbietet und damit zentrale Betriebsaufgaben (je nach Service-Stufe) stärker standardisiert. [62] Ebenso werden üblicherweise Funktionen wie Custom Domains und TLS-Absicherung unterstützt. [61]

Der Vorteil liegt vor allem im geringeren Betriebsaufwand, als Gegenstück dazu müssen Plattformlimits, Preisstufen und Rahmenbedingungen in die technische und wirtschaftliche Planung einbezogen werden. [61, 62]

2.8.6 Gegenüberstellung mit etablierten Cloud Anbietern

Cloud Computing ist eine zentrale Grundlage für den Betrieb moderner Microservice-Systeme, weil Ressourcen flexibel bereitgestellt, skaliert und gemessen werden können. [54] AWS und Azure bieten hierfür ausgereifte Plattformmechanismen und geben mit ihren Well-Architected-Leitlinien einen strukturierten Rahmen, um Architekturentscheidungen an Qualitätszielen auszurichten. [56–59]

World4You und Render zeigen, dass ähnliche Zielbilder auch außerhalb großer Hyperscaler realisierbar sind: Ein vServer bietet hohe Kontrolle, erfordert aber mehr Eigenbetrieb, eine PaaS-nahe Plattform reduziert den operativen Aufwand, bringt dafür stärkere Plattformabhängigkeiten mit. [7, 8, 61, 62]

2.8.7 Zusammenfassung

AWS und Microsoft Azure sind typische Beispiele für große Public-Cloud-Plattformen: Sie stellen weltweit verteilte Infrastruktur und standardisierte Dienste bereit, sodass Themen wie Skalierung, Verfügbarkeit und Betriebsautomatisierung häufig schon „mitgeliefert“ werden. Zur Orientierung bieten beide Anbieter Well-Architected-Frameworks, die helfen, Architekturentscheidungen systematisch an Qualitätszielen wie Zuverlässigkeit, Sicherheit und Kosten auszurichten.

Als praxisnahe Alternativen zeigen World4You und Render, dass Hosting auch außerhalb großer Hyperscaler sinnvoll umgesetzt werden kann. World4You (vServer) bietet viel Kontrolle und flexible Docker-Setups, verlangt dafür aber deutlich mehr Eigenbetrieb – etwa bei Updates, Backups sowie Monitoring und Logging. [7, 39, 52]

Render reduziert diesen Aufwand durch standardisierte Deployments und Managed PostgreSQL, dafür müssen Plattformlimits und Service-Stufen frühzeitig in die technische und wirtschaftliche Planung einfließen.

2.9 Einsatz sinnvoller Libraries und Technologien

Moderne Webanwendungen können heute Dinge, die früher fast nur mit Desktop-Programmen möglich waren – zum Beispiel PDFs erzeugen, QR- und Barcodes scannen oder Inhalte per Drag-and-Drop sortieren. Für solche Funktionen gibt es ausgereifte JavaScript-Werkzeuge, die direkt im Browser laufen. Diese Technologien werden auch in dem Bestellservice verwendet, um Abläufe und Funktionalitäten zu verbessern. Trotzdem müssen dabei praktische Grenzen wie Browserrechte (z. B. Kamerazugriff) und Performance auf mobilen Geräten mitgedacht werden. [63–68]

In den folgenden Abschnitten werden die in diesem Projekt verwendeten Bibliotheken (Libraries) sowie einige sicherheitsrelevante Techniken beschrieben. Ziel ist es, ihre Funktionsweise in dem Bestellservices nachvollziehbar zu machen.

2.9.1 PDF-Generierung mit jsPDF

Für die Erstellung von PDF-Dokumenten im Browser wird jsPDF eingesetzt. Die Bibliothek ermöglicht, Dokumente clientseitig zu generieren und als Datei bereitzustellen, ohne dass ein serverseitiger PDF-Generator zwingend erforderlich ist. Im Bestellservice ist dies insbesondere für Exportfunktionen relevant, bei denen strukturierte Daten (z. B. Bestell- oder Übersichtslisten) bereits im Frontend vorliegen und in ein transportier- bzw. archivierbares Format überführt werden sollen. [63, 64]

Technische Arbeitsweise

Technisch arbeitet jsPDF dokumentorientiert: Ein PDF wird über ein API schrittweise aufgebaut, wobei Inhalte über ein Koordinatensystem positioniert werden. Dadurch ist die Ausgabe kontrollierbar und reproduzierbar, erfordert aber zugleich eine bewusste Modellierung des Layouts (z. B. Seitenumbrüche, Zeilenlängen, Skalierung). Im Projektkontext ist das vorteilhaft, weil Exporte nicht „Screenshot-artig“ von der UI abhängen, sondern als eigenständige, konsistente Ausgabe erzeugt werden können. [63, 64]

2.9.2 QR-Code-Erkennung mit html5-qrcode

Für die clientseitige Erkennung von QR- und Barcodes wird eine browserbasierte Scan-Bibliothek verwendet, die Live-Scans über die Gerätekamera unterstützt. Damit lässt sich ein Scanvorgang direkt in den Webclient integrieren, was den Zugang erleichtert (keine Installation einer separaten App) und insbesondere für mobile Nutzungsszenarien geeignet ist. [66]

Technisch beruht der Scanprozess auf dem Zugriff auf einen Videostream und der fortlaufenden Auswertung einzelner Frames, bis ein Code erkannt und dekodiert werden kann. Für Stabilität und Performance sind Parameter wie Scanbereich und Abtastrate zentral, da sie den Ressourcenverbrauch und die Erkennungsrate je nach Endgerät deutlich beeinflussen können. Zusätzlich ist der Scanprozess an Browser-Berechtigungen gebunden, was eine saubere, transparente Anforderung und Nutzung der Kamera voraussetzt. [66]

2.9.3 Intuitive Bedienung mit Angular CDK Drag-and-Drop

Intuitive Bedienung mit Angular CDK Drag-and-Drop Um die Menüplanung im Bestellservice intuitiv und effizient zu gestalten, kommt das Drag-and-Drop-Modul des Angular Component Development Kits (CDK) zum Einsatz. Es ermöglicht, Elemente per Maus oder Touch-Eingabe zwischen Listen zu verschieben, innerhalb von Listen neu anzuordnen oder wieder zu entfernen. [65] Ein wesentlicher Vorteil liegt in der engen Integration in Angular: Benutzerinteraktionen werden konsistent auf die zugrunde liegenden Datenstrukturen übertragen. Dadurch entsteht ein stabiler, nachvollziehbarer Datenzustand, und gleichzeitig werden typische Eingabefehler reduziert, die bei rein formularbasierten Vorgehensweisen häufiger auftreten. [65] Im Projektkontext unterstützt dies insbesondere die flexible Zusammenstellung und Anpassung von Menüs durch berechtigte Nutzer. [65]

2.9.4 Absicherung der Verifikation und Authentifizierung via E-Mail

Die E-Mail-Adresse dient in vielen Webanwendungen sowohl als Identifikator als auch als Kommunikationskanal. Dieses Verfahren kommt auch in dieser Diplomarbeit zum Einsatz. Ohne Verifikation besteht das Risiko, dass Konten mit fremden oder fehlerhaften Adressen registriert werden, wodurch die Datenqualität sinkt und Missbrauch erleichtert wird. [21]

Ein verbreiteter Ansatz ist **Double-Opt-In**: Nach der Registrierung wird ein Konto zunächst eingeschränkt angelegt, erst der Aufruf eines Verifikationslinks aktiviert es vollständig. Damit wird der Besitz der E-Mail-Adresse bestätigt und Missbrauch erschwert. Für die Schutzwirkung ist wesentlich, dass sicherheitsrelevante Funktionen (z. B. Login oder Zugriff auf geschützte Ressourcen) erst nach erfolgreicher Verifikation freigeschaltet werden. [21]

Technisch basiert die Verifikation auf Tokens, die kryptografisch zufällig erzeugt, zeitlich begrenzt und nur einmal verwendbar sein müssen. OWASP betont hierbei insbesondere klare Ablaufzeiten und die konsequente Invalidierung nach der Nutzung durchzusetzen, um Brute-Force- und Replay-Szenarien zu reduzieren. [24] NIST ordnet Authentifizierungs- und Identitätsmechanismen zudem als zentrale Grundlage für sichere Zugriffskontrollen in Websystemen ein. [20, 21]

Beim Zurücksetzen von Passwörtern ist außerdem wichtig, dass die Anwendung nicht verrät, ob es zu einer E-Mail-Adresse ein Konto gibt. Sonst könnten Angreifer gezielt prüfen, welche Adressen registriert sind. Darum sollten Antworten auf Reset-Anfragen immer gleich formuliert und neutral gehalten sein – unabhängig davon, ob die Adresse im System existiert oder nicht. [26]

2.9.5 Zuverlässiger E-Mail-Versand: SMTP und API-basierte Alternativen

Der Versand von Verifikations- und Passwort-Reset-E-Mails wird über SMTP umgesetzt. SMTP ist asynchron konzipiert, das bedeutet, dass falls eine Zustellung kurzfristig nicht möglich ist, können Nachrichten zwischengespeichert und später erneut zugestellt werden, wodurch die Zuverlässigkeit erhöht wird. [68]

In der Praxis können jedoch organisatorische oder technische Rahmenbedingungen den SMTP-Versand einschränken. Dies ist beispielsweise in Schulen oder Unternehmen, der Fall. [68]. Diese Einschränkung betrifft auch unser Schulnetzwerk. Daher muss auf robuste Alternativen mit HTTPS-basiertem Versand z.B. über die Gmail API ausgewichen werden. Nur so kann dauerhaft ein stabiler E-Mail-Versand sichergestellt werden. Anders als bei SMTP erfolgt die Autorisierung bei der Gmail API über OAuth 2.0, wodurch die Versandrechte Token-basiert vergeben werden, ohne dass das Anwendungssystem Benutzerpasswörter speichern muss. OAuth 2.0 ist als Standard für eine solche Autorisierung etabliert und unterstützt eine kontrollierte Token-Vergabe. [67]

2.9.6 Zusammenfassung

Die im Projekt eingesetzten Bibliotheken ergänzen die Webanwendung um zentrale, praxisrelevante Funktionen: jsPDF ermöglicht reproduzierbare und kontrollierte PDF-Exporte, browserbasiertes QR-/Barcode-Scanning unterstützt scanbasierte Abläufe direkt im Webclient, und Angular CDK Drag-and-Drop verbessert die Menüplanung durch eine konsistente, effiziente Bearbeitung von Listen- und Strukturänderungen.

Sicherheitstechnisch ist die E-Mail-Verifikation ein wichtiger Baustein zur Absicherung und Überprüfung der Echtheit der User und zur Verbesserung der allgemeinen Datenqualität. Sie wird über Tokens umgesetzt, die zeitlich begrenzt sind und nur einmal verwendet werden können. Beim Passwort-Reset ist zusätzlich, aufgrund datenschutzrechtlicher Bestimmungen darauf zu achten, dass Antworten keine Rückschlüsse auf die Existenz anderer Konten zulassen.

Der E-Mail-Versand kann je nach Infrastruktur über SMTP oder alternativ, über ein API-basiertes Verfahren z.B. per Gmail API erfolgen. Eine genaue CORS-Policy ergänzt das Sicherheitskonzept und verhindert unbefugten Zugriff aus unterschiedlichsten Quellen.

3 Umsetzung

3.1 Grundstruktur des Projekts

3.2 Frontend-Entwicklung

3.3 Backend und API-Schnittstellen

3.4 PostgreSQL DB und DB-Zugriffe

3.5 Deployment mit World4You

4 Sicherung der Softwarequalität (Testing)

4.1 Frontend-Testing

4.2 Backend-Testing

5 Projektabschluss

5.1 Ergebnisse

5.2 Zukunftsausblick

5.3 Kurzanleitung zur Verwendung der Bestellapp

5.4 Danksagung

Abbildungsverzeichnis

Listingverzeichnis

Abkürzungsverzeichnis

Quellenverzeichnis

- [1] Proxmox Virtual Environment.
- [2] Amazon Web Services, Inc. (01.01.2026): Serverless Rechenleistung – AWS Fargate – AWS. <https://aws.amazon.com/de/fargate/>. Abgerufen am 01.01.2026.
- [3] Amazon Web Services, Inc. (01.01.2026): Hosted PostgreSQL – Amazon RDS für PostgreSQL – AWS. <https://aws.amazon.com/de/rds/postgresql/>. Abgerufen am 01.01.2026.
- [4] Gbowerman (23.12.2025): What Is Azure Database for PostgreSQL? <https://learn.microsoft.com/en-us/azure/postgresql/overview>. Abgerufen am 01.01.2026.
- [5] Msangapu-msft (09.09.2025): Scale Up Features and Capacities - Azure App Service. <https://learn.microsoft.com/en-us/azure/app-service/manage-scale-up>. Abgerufen am 01.01.2026.
- [6] Msangapu-msft (18.11.2025): Azure App Service Plans - Azure App Service. <https://learn.microsoft.com/en-us/azure/app-service/overview-hosting-plans>. Abgerufen am 01.01.2026.
- [7] World4You Internet Services GmbH (30.12.2025): Webhosting Österreich » günstige Webspace-Pakete. <https://www.world4you.com/hosting/web-hosting>. Abgerufen am 30.12.2025.
- [8] World4You Blog (30.12.2025): Hosting-News » Hole dir Wissen aus erster Hand. <https://www.world4you.com/blog/kategorie/webhosting/>. Abgerufen am 30.12.2025.
- [9] (19.02.2025): Angular - What is Angular? <https://v17.angular.io/guide/what-is-angular>. Abgerufen am 31.12.2025.
- [10](19.02.2025): Angular - Introduction to Angular concepts. <https://v17.angular.io/guide/architecture>. Abgerufen am 31.12.2025.
- [11](19.02.2025): Angular - Understanding communicating with backend services using HTTP. <https://v17.angular.io/guide/understanding-communicating-with-http>. Abgerufen am 31.12.2025.
- [12](31.12.2025): Documentation | NestJS - A progressive Node.js framework. <https://docs.nestjs.com/>. Abgerufen am 31.12.2025.
- [13]Documentation | NestJS - A progressive Node.js framework (31.12.2025): NestJS core concepts - Providers. <https://docs.nestjs.com/providers>. Abgerufen am 31.12.2025.
- [14](29.12.2025): Handbook - The TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>. Abgerufen am 31.12.2025.
- [15](29.12.2025): Documentation - TypeScript for JavaScript Programmers. <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>. Abgerufen am 31.12.2025.
- [16]Bundesministerium des Innern (2024): Datenschutzerklärung. https://www.bmi.bund.de/DE/service/datenschutz/datenschutz_node.html#doc9390964bodyText3. Abgerufen am 28.12.2025.

- [17] Landesbeauftragte für Datenschutz und Informationsfreiheit NRW (2025): Datenschutz | LDI - Landesbeauftragte für Datenschutz und Informationsfreiheit Nordrhein-Westfalen. <https://www.ldi.nrw.de/datenschutz>. Abgerufen am 28.12.2025.
- [18] Europäische Union | EUR-Lex (2016): DSGVO - EU: Regulation - 2016/679 - EN - gdpr - EUR-Lex. DSGVO. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>. Abgerufen am 28.12.2025.
- [19] European Commission (28.12.2025): Principles of the GDPR. https://commission.europa.eu/law/law-topic/data-protection/rules-business-and-organisations/principles-gdpr_en. Abgerufen am 28.12.2025.
- [20] NIST (2025): NIST Special Publication 800-63B. <https://pages.nist.gov/800-63-3/sp800-63b.html>. Abgerufen am 28.12.2025.
- [21] (2025): Authentication - OWASP Cheat Sheet Series. https://cheatsheet-series.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Abgerufen am 28.12.2025.
- [22] Justinha (2023): Microsoft Entra multifactor authentication overview - Microsoft Entra ID. How it works. <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-mfa-howitworks>. Abgerufen am 28.12.2025.
- [23] M. Jones, DH (2012): Bearer Token Usage - Internet Engineering Task Force.
- [24] M. Jones, ea (2015): RFC 7519: JSON Web Token (JWT). <https://www.rfc-editor.org/rfc/rfc7519>. Abgerufen am 28.12.2025.
- [25] R. T. Fielding, R. N. Taylor (2002): Architectural styles and the design of network-based software architectures | Guide books. <https://dl.acm.org/doi/10.1145/514183.514185>. Abgerufen am 28.12.2025.
- [26] (24.12.2025): Forgot Password - OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html. Abgerufen am 28.12.2025.
- [27] (2025): HTTP-Cors-Protocol. Abgerufen am 28.12.2025.
- [28] portSwigger Web Security Academy (30.12.2025): What is CORS (cross-origin resource sharing)? Tutorial & Examples | Web Security Academy. <https://portswigger.net/web-security/cors>. Abgerufen am 30.12.2025.
- [29] E. F. Codd (1970): A relational model of data for large shared data banks. <https://dl.acm.org/doi/epdf/10.1145/362384.362685>. Abgerufen am 30.12.2025.
- [30] Adrienne Watt (2014): Chapter 7 The Relational Data Model. Database Design - 2nd Edition.
- [31] PostgreSQL Documentation (2025): The SQL Language. <https://www.postgresql.org/docs/current/sql.html>. Abgerufen am 30.12.2025.
- [32] Adrienne Watt (2014): Chapter 12 Normalization. Database Design - 2nd Edition.
- [33] PostgreSQL Documentation (2025): Transactions. <https://www.postgresql.org/docs/current/tutorial-transactions.html>. Abgerufen am 30.12.2025.
- [34] PostgreSQL Documentation (2025): Chapter 28. Reliability and the Write-Ahead Log. <https://www.postgresql.org/docs/current/wal.html>. Abgerufen am 30.12.2025.
- [35] PostgreSQL Documentation (2025): A Brief History of PostgreSQL. <https://www.postgresql.org/docs/current/history.html>. Abgerufen am 30.12.2025.

- [36]Chauhan, C (2017): Understanding the PostgreSQL Architecture. <https://severalnines.com/blog/understanding-postgresql-architecture/>. Abgerufen am 30.12.2025.
- [37]Bhupathi, S (10.07.2024): PostgreSQL MVCC. OPENSOURCE DB PRIVATE LIMITED.
- [38]Dirk, M (2014): Docker: Lightweight Linux Containers for Consistent Development and Deployment | Linux Journal. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>. Abgerufen am 30.12.2025.
- [39]Docker Documentation (27.12.2025): What is Docker? <https://docs.docker.com/get-started/docker-overview/>. Abgerufen am 30.12.2025.
- [40]Docker Documentation (2024): Networking. <https://docs.docker.com/engine/network/>. Abgerufen am 30.12.2025.
- [41]Docker Documentation (27.12.2025): Volumes. <https://docs.docker.com/engine/storage/volumes/>. Abgerufen am 30.12.2025.
- [42]Atlassian (19.12.2025): Microservices Architecture | Atlassian. <https://www.atlassian.com/microservices/microservices-architecture>. Abgerufen am 30.12.2025.
- [43]Chris Richardson (30.12.2025): Microservices Pattern: Microservice Architecture pattern. <https://microservices.io/patterns/microservices.html>. Abgerufen am 30.12.2025.
- [44]Claytonsiemens77 (10.12.2025): Microservices Architecture Style - Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>. Abgerufen am 30.12.2025.
- [45]Heinrichs, T (01.02.2023): Orchestration vs Choreography. Camunda.
- [46]Olaf Zimmermann, Mirko Stocker, Daniel Lübke, Cesare Pautasso, Uwe Zdun (2019): Introduction to Microservice API Patterns (MAP).
- [47](10.09.2024): InterService Communication in Microservices. GeeksforGeeks.
- [48]Claytonsiemens77 (03.12.2025): Cloud Design Patterns - Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/patterns/>. Abgerufen am 30.12.2025.
- [49](24.12.2025): JSON Web Token for Java - OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html. Abgerufen am 28.12.2025.
- [50]Kubernetes (23.11.2025): Deployments. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>. Abgerufen am 31.12.2025.
- [51]Kubernetes (31.12.2025): Services, Load Balancing, and Networking. <https://kubernetes.io/docs/concepts/services-networking/>. Abgerufen am 31.12.2025.
- [52]Bundesamt für Sicherheit in der Informationstechnik (2023): IT-Grundschutz-Kompendium. https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/IT-Grundschutz/IT-Grundschutz-Kompendium/it-grundschutz-kompendium_node.html. Abgerufen am 28.12.2025.
- [53]Claytonsiemens77 (09.12.2025): Saga Design Pattern - Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/patterns/saga>. Abgerufen am 30.12.2025.
- [54]Mell, PM, Grance, T (2011): The NIST definition of cloud computing.

- [55]Fang, Liu, et. al. (2011): <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf>. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf>. Abgerufen am 30.12.2025.
- [56]AWS (2024): Was ist Webhosting? https://aws.amazon.com/what-is/webhosting/?trk=faq_card. Abgerufen am 30.12.2025.
- [57]AWS (2024): AWS Well-Architected Framework - AWS Well-Architected Framework. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>. Abgerufen am 30.12.2025.
- [58]Microsoft (20.11.2025): How to use the Azure Well-Architected Framework documentation - Microsoft Azure Well-Architected Framework. <https://learn.microsoft.com/en-us/azure/well-architected/what-is-well-architected-framework>. Abgerufen am 30.12.2025.
- [59]PageWriter-MSFT (20.11.2025): How to use the Azure Well-Architected Framework documentation - Microsoft Azure Well-Architected Framework. <https://learn.microsoft.com/en-us/azure/well-architected/what-is-well-architected-framework>. Abgerufen am 30.12.2025.
- [60]Claytonsiemens77 (30.10.2025): Azure Application Architecture Fundamentals - Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/>. Abgerufen am 30.12.2025.
- [61]Render (30.12.2025): Docker on Render. <https://render.com/docs/docker>. Abgerufen am 30.12.2025.
- [62]Render (30.12.2025): Create and Connect to Render Postgres. <https://render.com/docs/postgresql-creating-connecting>. Abgerufen am 30.12.2025.
- [63]parallax (2025): jsPDF - Documentation. <https://parallax.github.io/jsPDF/docs/jsPDF.html>. Abgerufen am 30.12.2025.
- [64]parallax (2018): Client-side JavaScript PDF generation for everyone. <https://github.com/parallax/jsPDF>. Abgerufen am 30.12.2025.
- [65]Team, A (18.12.2025): Drag and drop Angular. <https://angular.dev/guide/drag-drop>. Abgerufen am 30.12.2025.
- [66]Kevin (2025): Building an HTML5 QR Code & Barcode Scanner in JavaScript. <https://scanbot.io/techblog/html5-barcode-scanner-tutorial/>. Abgerufen am 30.12.2025.
- [67](11.12.2025): Gmail API - Node.js quickstart Google for Developers. <https://developers.google.com/workspace/gmail/api/quickstart/nodejs>. Abgerufen am 30.12.2025.
- [68]Klensin, J (2008): <https://www.rfc-editor.org/rfc/pdf/rfc5321.txt.pdf>. <https://www.rfc-editor.org/rfc/pdf/rfc5321.txt.pdf>. Abgerufen am 30.12.2025.